

# Rab Bashir

## Counsel X

 Quick Submit Quick Submit COMSATS Institute of Information Technology

---

### Document Details

Submission ID

trn:oid::1:3529878485

Submission Date

Apr 7, 2026, 7:27 PM GMT+5

Download Date

Apr 7, 2026, 7:33 PM GMT+5

File Name

FYP\_Latex\_Templete\_8\_1.pdf

File Size

1.2 MB

89 Pages

18,814 Words

111,100 Characters

## 41% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

**Caution: Review required.**

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

### Detection Groups



**105 AI-generated only 41%**

Likely AI-generated text from a large-language model.



**0 AI-generated text that was AI-paraphrased 0%**

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

#### Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

### Frequently Asked Questions

#### How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (\*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

#### What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



# COMSATS University Islamabad, Vehari Campus, Pakistan

Department of Computer Science

## Counsel-X: An Integrated Web-Based Platform for Student Counseling and Mental Well being



A Thesis / Software Requirements Specification (SRS) / Software Design  
Document (SDD) Submitted in Partial Fulfillment of the Requirements  
for the Degree of

## Bachelor of Science in Computer Science

### Submitted By

Samran Zafar    FA22-BCS-044  
Asad Naeem    FA22-BCS-013

### Supervisor

Dr Umar Rasheed

Session: Spring 2022-2026

# Dedication

I dedicate this work to my parents, whose unwavering support, sacrifices, and prayers have been the foundation of my success. Their encouragement, patience, and belief in my abilities have guided me throughout this journey.

I also dedicate this to my teachers and mentors who inspired me to pursue knowledge with dedication and integrity.

# Acknowledgement

All praise and gratitude are due for the successful completion of this project. The authors would like to express their sincere appreciation to their supervisor for continuous guidance, valuable suggestions, and constructive feedback throughout the development of this thesis. The supervision and mentorship provided significant support in completing this work successfully.

Special thanks are extended to the faculty members of the Department of Computer Science for their academic support and encouragement during the degree program.

The authors are also deeply grateful to their parents and families for their unwavering support, motivation, and prayers, which have been a constant source of strength.

Finally, appreciation is extended to everyone who directly or indirectly contributed to the successful completion of this project.

# Certificate of Approval

This is to certify that the Final Year Project titled “**Counsel-X**” has been developed by **Samran Zafar (FA22-BCS-044)** and **Asad Naeem (FA22-BCS-013)** under the supervision of **Dr Umar Rasheed** and that in his opinion, the project is adequate in scope and quality for the award of the degree of Bachelor of Science in Computer Science.

---

Supervisor

---

External Examiner

---

Head of Department  
Department of Computer Science

# Plagiarism and AI Content Certificate

Date: \_\_\_\_\_

It is hereby certified that the similarity index (as generated by the Turnitin Originality Report) of the [Report Type: SRS / SDD / Thesis / Final Year Project Report] submitted by:

- Samran Zafar FA22-BCS-044
- Asad Naeem FA22-BCS-013

titled:

**“Counsel-X”**

is [XX%] which is within the permissible limits as defined by the Higher Education Commission (HEC) rules and regulations. The similarity index report is attached herewith.

Furthermore, the AI-generated content detection index (if applicable as per institutional policy) is reported as [XX%]. The report has been reviewed, and the content is found to comply with university guidelines regarding ethical use of Artificial Intelligence tools.

The report is recommended for submission.

**Supervisor Name:** \_\_\_\_\_

**Designation:** \_\_\_\_\_

**Signature:** \_\_\_\_\_

**Department:** Department of Computer Science

**University:** COMSATS University Islamabad, Vehari Campus

## Executive Summary

The Counsel-X initiative is a sophisticated web-based system created to eliminate the imbalance of those who would like to help others and those who want help or guidance. Modern education has placed students under tremendous pressure due to the demands of academia and the many forms of stress that accompany these demands such as: excessive exam stress, career uncertainty, depression, anxiety etc. Traditional means of guiding students do not provide solutions to the complex issues students experience today. Many traditional counseling centers have long wait times, are only available during limited times and may lack sufficient privacy for a person to feel comfortable seeking assistance from a counselor. Counsel-X removes the obstacles of traditional counseling and provides a digital sanctuary a private, secure and immediate online connection to qualified professionals 24 hours a day, 7 days a week. Counsel-X was created in response to the alarming increase in the number of students worldwide who suffer from anxiety and the lack of a comprehensive, digital-first approach to addressing this issue. There are some general means of communicating with friends and family; however, there is no means of providing support to a student through a counselor based on their individual needs and the emotional component of the counselor-student relationship. The principle behind Counsel-X is to address the issue of Accessibility Gap, which includes, but is not limited to, the distance that remote students travel to see an off-campus therapist, the rising cost and the "bottleneck" of administrative work at universities when trying to access counseling through a university-managed portal. Counsel-X leverages the MERN stack (MongoDB, Express.js, React, Node.js) to deliver an extremely high-performance application that keeps data private and secure via full encryption while ensuring real-time human interaction remains possible with maximum responsiveness. The technological framework enables Counsel-X to handle a large number of concurrent users, providing a scalable resource to all educational institutions. The platform's approach to user experience is focused on specialized, role-specific experiences that help give users access to a range of unique tools. Each different type of user can use a dashboard that is tailored to their respective needs. Some of the unique features of this system are its ability to utilize Socket.IO to connect users through a low-latency real-time chat system that simulates the intimacy of a face-to-face meeting while still providing users with the comfort of a digital interface, as well as the use of the Stripe API

to provide financial transparency and ease of use by processing secure per-session payments and automated invoicing in the process. The platform also includes an intelligent scheduling engine that automates and eliminates the possibility of double bookings and has a curated digital library. The library also includes educational videos, self care books, and career guides for students to use at all times and to continue receiving value from the system outside of the real-time sessions. Counsel-X will create a more resilient, informed, and mentally healthy student body through its anonymous and user-friendly interface, which will work to destigmatize help seeking behavior by enabling students to see guidance as a proactive means of achieving success rather than merely a last resort. Furthermore, Counsel-X will allow colleges and universities to track student wellness patterns and enhance retention rates in a manner that does not impose additional administrative burdens on those institutions. Therefore, this initiative is an important contribution to the emerging market at the intersection of "EdTech" and "MedTech." Counsel-X demonstrates the ability of advanced web technologies to be combined with genuine human compassion to produce an integrated support system that protects the academic, vocational, and social futures of the student of today.

# Abbreviations

| Abbreviation | Full Form                           |
|--------------|-------------------------------------|
| SRS          | Software Requirements Specification |
| SDD          | Software Design Document            |
| API          | Application Programming Interface   |
| UI           | User Interface                      |
| DB           | Database                            |

# Table of Contents

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| Executive Summary . . . . .                                             | v         |
| List of Tables . . . . .                                                | xiii      |
| List of Figures . . . . .                                               | xv        |
| <b>1 Introduction</b>                                                   | <b>1</b>  |
| 1.1 Introduction . . . . .                                              | 1         |
| 1.2 Vision Statement . . . . .                                          | 2         |
| 1.3 Related System Analysis . . . . .                                   | 2         |
| 1.4 Project Deliverables . . . . .                                      | 3         |
| 1.5 System Limitations / Constraints . . . . .                          | 4         |
| 1.6 Tools and Technologies . . . . .                                    | 4         |
| 1.7 Relevance to Course Modules . . . . .                               | 4         |
| 1.7.1 Programming Fundamentals . . . . .                                | 4         |
| 1.7.2 Data Structures and Algorithms . . . . .                          | 5         |
| 1.7.3 Database Management Systems . . . . .                             | 5         |
| 1.7.4 Software Engineering . . . . .                                    | 6         |
| 1.7.5 Other Relevant Courses . . . . .                                  | 6         |
| 1.8 Chapter Summary . . . . .                                           | 6         |
| <b>2 Problem Definition</b>                                             | <b>7</b>  |
| 2.1 Problem Statement . . . . .                                         | 7         |
| 2.2 Proposed Solution Overview . . . . .                                | 7         |
| 2.3 Objectives of the Proposed System . . . . .                         | 7         |
| 2.4 Scope of the System . . . . .                                       | 8         |
| 2.5 System Architecture and Modules . . . . .                           | 8         |
| 2.5.1 Module 1: Profile Management . . . . .                            | 8         |
| 2.5.2 Module 2: Communication Module . . . . .                          | 9         |
| 2.5.3 Module 3: Session and Booking Management Module . . . . .         | 9         |
| 2.5.4 Module 4: Administrative and Resource Management Module . . . . . | 10        |
| 2.5.5 Module 5: Financial and Notification Services Module . . . . .    | 10        |
| 2.6 Assumptions and Dependencies . . . . .                              | 11        |
| 2.7 Chapter Summary . . . . .                                           | 11        |
| <b>3 Requirement Analysis</b>                                           | <b>12</b> |
| 3.1 Introduction to Requirement Analysis . . . . .                      | 12        |
| 3.2 Requirement Elicitation Techniques . . . . .                        | 12        |
| 3.3 User Classes and Characteristics . . . . .                          | 12        |
| 3.4 System Overview . . . . .                                           | 13        |

|          |                                                           |    |
|----------|-----------------------------------------------------------|----|
| 3.5      | Use Case Model . . . . .                                  | 13 |
| 3.5.1    | Use Case Diagrams . . . . .                               | 13 |
| 3.5.1.1  | Login Use Case Diagram . . . . .                          | 13 |
| 3.5.1.2  | Student Use Case Diagram . . . . .                        | 14 |
| 3.5.1.3  | Counsellor Use Case Diagram . . . . .                     | 15 |
| 3.5.1.4  | Administrator Use Case Diagram . . . . .                  | 16 |
| 3.5.2    | Use Case Descriptions . . . . .                           | 18 |
| 3.5.2.1  | UC-1: Login / Authentication . . . . .                    | 18 |
| 3.5.2.2  | UC-2: Manage Profile . . . . .                            | 19 |
| 3.5.2.3  | UC-3: Browse Counselors . . . . .                         | 19 |
| 3.5.2.4  | UC-4: Schedule Session . . . . .                          | 20 |
| 3.5.2.5  | UC-5: Live Chat Session . . . . .                         | 20 |
| 3.5.2.6  | UC-6: Access Digital Library . . . . .                    | 21 |
| 3.5.2.7  | UC-7: Purchase Course . . . . .                           | 21 |
| 3.5.2.8  | UC-8: Verify Counselor . . . . .                          | 22 |
| 3.5.2.9  | UC-9: Manage User Accounts . . . . .                      | 22 |
| 3.5.2.10 | UC-10: Manage Library Content . . . . .                   | 23 |
| 3.5.3    | Event-Response Specifications . . . . .                   | 23 |
| 3.5.3.1  | EV-1: Login Attempt Response . . . . .                    | 23 |
| 3.5.3.2  | EV-2: Payment Submission Response . . . . .               | 24 |
| 3.6      | Functional Requirements . . . . .                         | 24 |
| 3.6.1    | Module 1: Authentication and Access . . . . .             | 24 |
| 3.6.2    | Module 2: Session & Communication Management . . . . .    | 25 |
| 3.6.2.1  | FR-2.1 Live Chat Session . . . . .                        | 25 |
| 3.6.2.2  | :FR-2.2 : Session Scheduling . . . . .                    | 25 |
| 3.6.3    | Module 3: Profile and Resource Management . . . . .       | 26 |
| 3.6.3.1  | FR-3.1 Profile Management . . . . .                       | 26 |
| 3.6.3.2  | Module 5: FR-3.2 Resource Management . . . . .            | 26 |
| 3.6.4    | Module 4: Payment and Administrative Management . . . . . | 27 |
| 3.6.4.1  | FR-4.1 Payment Processing . . . . .                       | 27 |
| 3.6.4.2  | FR-4.2: Administrative Management . . . . .               | 27 |
| 3.7      | Non-Functional Requirements . . . . .                     | 27 |
| 3.7.1    | Performance Requirements . . . . .                        | 27 |
| 3.7.2    | Security Requirements . . . . .                           | 28 |
| 3.7.3    | Reliability Requirements . . . . .                        | 28 |
| 3.7.4    | Usability Requirements . . . . .                          | 28 |
| 3.7.5    | Scalability Requirements . . . . .                        | 28 |
| 3.7.6    | Maintainability Requirements . . . . .                    | 29 |
| 3.8      | External Interface Requirements . . . . .                 | 29 |
| 3.8.1    | User Interface Requirements . . . . .                     | 29 |
| 3.8.2    | Hardware Interface Requirements . . . . .                 | 29 |
| 3.8.3    | Software Interface Requirements . . . . .                 | 29 |
| 3.8.4    | Communication Interface Requirements . . . . .            | 30 |
| 3.8.5    | Database Interface Requirements . . . . .                 | 30 |
| 3.8.6    | API Interface Requirements . . . . .                      | 30 |

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| 3.9      | Assumptions and Dependencies . . . . .                  | 30        |
| 3.10     | Requirement Traceability Matrix . . . . .               | 31        |
| <b>4</b> | <b>Software Design</b>                                  | <b>32</b> |
| 4.1      | Introduction to Software Design . . . . .               | 32        |
| 4.2      | Design Methodology and Software Process Model . . . . . | 32        |
| 4.2.1    | Design Methodology . . . . .                            | 32        |
| 4.2.2    | Software Process Model . . . . .                        | 33        |
| 4.3      | System Overview . . . . .                               | 33        |
| 4.4      | Architectural Design . . . . .                          | 34        |
| 4.5      | Design Models . . . . .                                 | 36        |
| 4.5.1    | Activity Diagrams . . . . .                             | 36        |
| 4.5.1.1  | Module 1: User Management & Authentication . . . . .    | 36        |
| 4.5.1.2  | Module 2: Session Booking & Real-Time Chat . . . . .    | 38        |
| 4.5.1.3  | Module 3: Admin Module . . . . .                        | 40        |
| 4.5.2    | Class Diagram . . . . .                                 | 40        |
| 4.5.3    | Sequence Diagrams . . . . .                             | 42        |
| 4.5.3.1  | Sequence Diagram of Counselor . . . . .                 | 42        |
| 4.5.3.2  | Sequence Diagram of Student . . . . .                   | 43        |
| 4.5.3.3  | Sequence Diagram of overall System . . . . .            | 43        |
| 4.5.4    | State Transition Diagrams . . . . .                     | 45        |
| 4.5.4.1  | User Authentication State Diagram . . . . .             | 45        |
| 4.5.4.2  | Payment Lifecycle State Diagram . . . . .               | 46        |
| 4.5.4.3  | Counseling Session State Diagram . . . . .              | 47        |
| 4.5.4.4  | Resource Library Management State Diagram . . . . .     | 48        |
| 4.6      | Data Flow Diagrams . . . . .                            | 49        |
| 4.6.1    | Level 1 Data Flow Diagram . . . . .                     | 49        |
| 4.6.2    | Level 2 Data Flow Diagram: Chat Processing . . . . .    | 49        |
| 4.7      | Data Design . . . . .                                   | 50        |
| 4.7.1    | Entity Relationship Diagram (ERD) . . . . .             | 50        |
| 4.7.2    | Database Architecture . . . . .                         | 51        |
| 4.8      | Database Schema & Data Dictionary . . . . .             | 51        |
| 4.8.1    | Data Dictionary . . . . .                               | 52        |
| 4.8.1.1  | User Table Structure . . . . .                          | 52        |
| 4.8.1.2  | Session Table Structure . . . . .                       | 52        |
| 4.8.1.3  | Payments Table Structure . . . . .                      | 52        |
| 4.8.1.4  | Resource Library Table Structure . . . . .              | 53        |
| 4.8.1.5  | Chat Messages Table Structure . . . . .                 | 53        |
| 4.8.1.6  | Reviews Table Structure . . . . .                       | 53        |
| 4.8.1.7  | Counselor Profiles Table Structure . . . . .            | 53        |
| 4.8.1.8  | Course Categories Table Structure . . . . .             | 54        |
| 4.9      | Database Tables / Collections . . . . .                 | 54        |
| 4.9.1    | User Collection / Table . . . . .                       | 54        |
| 4.9.2    | Core Data Collection . . . . .                          | 54        |
| 4.9.3    | Relationships Between Tables . . . . .                  | 55        |
| 4.9.4    | Security and Data Integrity . . . . .                   | 55        |

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| 4.9.5 Scalability Considerations . . . . .                         | 55        |
| Chapter Summary . . . . .                                          | 56        |
| <b>5 Implementation . . . . .</b>                                  | <b>57</b> |
| 5.1 Development Environment . . . . .                              | 57        |
| 5.1.1 Software Specifications . . . . .                            | 57        |
| 5.1.2 Hardware Specifications . . . . .                            | 57        |
| 5.2 Core Module Implementation . . . . .                           | 58        |
| 5.2.1 Module 1: Admin Module . . . . .                             | 58        |
| 5.2.2 Module 2: Student Module . . . . .                           | 58        |
| 5.2.3 Module 3: Counselor Module . . . . .                         | 58        |
| 5.2.4 Module 4: Chat Module (Real-time Core) . . . . .             | 59        |
| 5.3 Algorithm Implementation . . . . .                             | 59        |
| 5.3.1 Algorithm 1: JWT-Based Authentication . . . . .              | 59        |
| 5.3.2 Algorithm 2: 45-Minute Session Control (Socket.io) . . . . . | 59        |
| 5.4 External APIs / SDKs . . . . .                                 | 60        |
| 5.5 User Interface Implementation . . . . .                        | 60        |
| 5.5.1 UI Design Approach . . . . .                                 | 60        |
| 5.5.2 Screen-wise Implementation . . . . .                         | 60        |
| 5.5.2.1 Screen 1: Login Screen . . . . .                           | 60        |
| 5.5.2.2 Screen 2: Registration Screen . . . . .                    | 60        |
| 5.5.2.3 Screen 3: Student Dashboard . . . . .                      | 61        |
| 5.5.2.4 Screen 4: Counselor Selection Grid . . . . .               | 61        |
| 5.5.2.5 Screen 5: Appointment Booking (Calendar) . . . . .         | 61        |
| 5.5.2.6 Screen 6: Checkout / Payment Screen . . . . .              | 61        |
| 5.5.2.7 Screen 7: Live Chat Interface . . . . .                    | 61        |
| 5.5.2.8 Screen 8: Resource Library . . . . .                       | 61        |
| 5.5.3 Navigation Flow Diagram . . . . .                            | 62        |
| 5.5.4 Responsiveness and Performance . . . . .                     | 62        |
| 5.6 Deployment . . . . .                                           | 62        |
| Chapter Summary . . . . .                                          | 62        |
| <b>6 Testing and Evaluation . . . . .</b>                          | <b>63</b> |
| 6.1 Testing Strategy . . . . .                                     | 63        |
| 6.2 Unit Testing . . . . .                                         | 63        |
| 6.2.1 UT-01: User Registration . . . . .                           | 63        |
| 6.2.2 UT-02: Login & Authentication . . . . .                      | 64        |
| 6.2.3 UT-03: Resource Upload (Admin) . . . . .                     | 64        |
| 6.2.4 UT-04: Session Booking (Student) . . . . .                   | 64        |
| 6.2.5 UT-05: Chat Room Access Logic . . . . .                      | 64        |
| 6.3 Integration Testing . . . . .                                  | 64        |
| 6.4 System Testing . . . . .                                       | 65        |
| 6.5 Performance Testing . . . . .                                  | 65        |
| 6.6 Security Testing . . . . .                                     | 66        |
| 6.7 User Acceptance Testing (UAT) . . . . .                        | 66        |
| 6.8 Testing Summary . . . . .                                      | 66        |

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>7 Conclusion and Future Work</b>                | <b>68</b> |
| 7.1 Conclusion . . . . .                           | 68        |
| 7.2 Future Work . . . . .                          | 68        |
| 7.3 Limitations . . . . .                          | 69        |
| <b>Appendices</b>                                  | <b>71</b> |
| Appendix A – Turnitin Similarity Report . . . . .  | 71        |
| Appendix B – AI Writing Detection Report . . . . . | 72        |
| Appendix C – Source Code . . . . .                 | 73        |

# List of Tables

|      |                                                      |    |
|------|------------------------------------------------------|----|
| 1.1  | Comparative Analysis of Existing Systems . . . . .   | 3  |
| 1.2  | Tools and Technologies Used in the Project . . . . . | 5  |
| 3.1  | User Classes and Characteristics . . . . .           | 13 |
| 3.2  | UC-1: Login / Authentication . . . . .               | 18 |
| 3.3  | UC-2: Manage Profile . . . . .                       | 19 |
| 3.4  | UC-3: Browse Counselors . . . . .                    | 19 |
| 3.5  | UC-4: Schedule Session . . . . .                     | 20 |
| 3.6  | UC-5: Live Chat Session . . . . .                    | 20 |
| 3.7  | UC-6: Access Digital Library . . . . .               | 21 |
| 3.8  | UC-7: Purchase Course . . . . .                      | 21 |
| 3.9  | UC-8: Verify Counselor . . . . .                     | 22 |
| 3.10 | UC-9: Manage User Accounts . . . . .                 | 22 |
| 3.11 | UC-10: Manage Library Content . . . . .              | 23 |
| 3.12 | EV-1: Login Attempt Response . . . . .               | 23 |
| 3.13 | EV-2: Payment Submission Response . . . . .          | 24 |
| 3.14 | FR-01: User Login Functionality . . . . .            | 24 |
| 3.15 | FR-03: Time-Bounded Chat Session . . . . .           | 25 |
| 3.16 | FR-02: Session Scheduling . . . . .                  | 25 |
| 3.17 | FR-06: Counselor Profile Management . . . . .        | 26 |
| 3.18 | FR-07: Digital Library Management . . . . .          | 26 |
| 3.19 | FR-04: Payment Processing . . . . .                  | 27 |
| 3.20 | FR-05: Administrative User Management . . . . .      | 27 |
| 3.21 | Requirement Traceability Matrix . . . . .            | 31 |
| 4.1  | User Table Structure . . . . .                       | 52 |
| 4.2  | Session Table Structure . . . . .                    | 52 |
| 4.3  | Payments Table Structure . . . . .                   | 52 |
| 4.4  | Resource Library Table Structure . . . . .           | 53 |
| 4.5  | Chat Messages Table Structure . . . . .              | 53 |
| 4.6  | Reviews Table Structure . . . . .                    | 53 |
| 4.7  | Counselor Profiles Table Structure . . . . .         | 53 |
| 4.8  | Course Categories Table Structure . . . . .          | 54 |
| 4.9  | User Table Detailed Structure . . . . .              | 54 |
| 5.1  | External APIs Used . . . . .                         | 60 |
| 5.2  | Deployment Details . . . . .                         | 62 |

|     |                                                |    |
|-----|------------------------------------------------|----|
| 6.1 | Unit Test Case – User Registration . . . . .   | 63 |
| 6.2 | Unit Test Case – Login Module . . . . .        | 64 |
| 6.3 | Unit Test Case – Resource Management . . . . . | 64 |
| 6.4 | Unit Test Case – Appointment Booking . . . . . | 64 |
| 6.5 | Unit Test Case – 45-Minute Window . . . . .    | 65 |
| 6.6 | Integration Testing Summary . . . . .          | 65 |
| 6.7 | Performance Metrics . . . . .                  | 66 |
| 6.8 | User Feedback Summary . . . . .                | 67 |

# List of Figures

|      |                                                        |    |
|------|--------------------------------------------------------|----|
| 2.1  | System Architecture Diagram . . . . .                  | 9  |
| 3.1  | Login Use Case Diagram . . . . .                       | 14 |
| 3.2  | Student Use Case Diagram . . . . .                     | 15 |
| 3.3  | Counsellor Use Case Diagram . . . . .                  | 16 |
| 3.4  | Administrator Use Case Diagram . . . . .               | 17 |
| 4.1  | Context Diagram of Counsel-X . . . . .                 | 34 |
| 4.2  | System Architecture Diagram . . . . .                  | 35 |
| 4.3  | Activity Diagram – User Module . . . . .               | 37 |
| 4.4  | Activity Diagram – Core Functional Module . . . . .    | 39 |
| 4.5  | Activity Diagram – Admin Operations . . . . .          | 40 |
| 4.6  | Class Diagram for Counsel-X . . . . .                  | 41 |
| 4.7  | Sequence Diagram of Counselor . . . . .                | 42 |
| 4.8  | Sequence Diagram of Student . . . . .                  | 43 |
| 4.9  | Sequence Diagram of Overall System . . . . .           | 44 |
| 4.10 | User Authentication State Transition Diagram . . . . . | 45 |
| 4.11 | Payment Lifecycle State Transition Diagram . . . . .   | 46 |
| 4.12 | Counseling Session State Transition Diagram . . . . .  | 47 |
| 4.13 | Resource Library State Transition Diagram . . . . .    | 48 |
| 4.14 | Level 1 Data Flow Diagram . . . . .                    | 49 |
| 4.15 | Level 2 Data Flow Diagram . . . . .                    | 50 |
| 4.16 | Entity Relation Diagram for Counsel-X . . . . .        | 50 |
| 4.17 | Database Schema Diagram . . . . .                      | 51 |

# Chapter 1

## Introduction

### 1.1 Introduction

As the modern academic world changes drastically due to COVID-19 and changes in technology, students need help managing their mental health, personal development, and their mental health. Research shows that students' level of anxiety and stress has greater than doubled in the last ten years and that this increase due to both competitive academic environments and the uncertainty surrounding the current global employment market. Many students continue to struggle to access the essential mental health services due to geographical access, a lack of qualified and experienced practitioners, and limited service hours that conflict with their academic schedule. In many cases, counseling is dependent on in-person meetings at an office, which do not adequately meet the needs of the digitally educated student population who require increased flexibility, anonymity, and immediate availability of services.

The transition to remote locations due to the COVID-19 pandemic and subsequent advancement of technology has significantly changed expectations for distance services in education. Most of the industry today is still manually scheduled appointments or utilizing simple university portals with limited ability to interact with students in real-time. While there are localized traditional or telehealth models that provide some access to care, they experience significant administrative bottlenecks, limited scalability, and no means of integrated payment or tracking. Additionally, many students avoid using traditional providers due to the continuing stigma connected with accessing mental health care.

This project is involved in addressing the systemic inefficiencies in delivering effective support to students through the CounselX platform - an all-inclusive, enterprise-level, web-based platform for democratizing access to professional guidance services. CounselX has been established to provide a digital bridge for students to connect with qualified counselors and schedule secure, encrypted video sessions anytime and from any location. Unlike other provider platforms that focus primarily on clinical therapy, CounselX provides holistic supports across multiple domains, including academic course selection, career trajectory planning, mental health advocacy, and relationship counseling. By eliminating the boundaries of in-person supporting sessions, CounselX provides a safe space for students to get the help they need at their convenience; thus eliminating the barriers that exist between being able to identify a problem and receiving assistance.

CounselX's platform architecture has been developed on a high performance platform to offer a personalized user experience through dedicated dashboards for students, counselors and administrators. Key functionalities to support the operation of the platform such as secure payment processing through Stripe, automated email reminders and real-time communication modules using WebSockets have been included to improve operational efficiency. The MERN stack (MongoDB, Express.js, React and Node.js) will be used to develop CounselX, which will utilize non-blocking I/O and component-based architecture to maintain high level performance, consistency and scalability. By utilizing this technology, .

## 1.2 Vision Statement

Counsel-X's vision is to revolutionise global student support so that all students can receive academic and personal assistance without fear of being alone. We will create the “digital sanctuary” – a secure, safe, and nurturing virtual space where it's as easy to get professional assistance as it is to use modern digital technology. This vision is based on the idea that high-quality, affordable counselling shouldn't only be available in urban centres or to those with large amounts of money. Our goal is to create the universal bridge that connects skill with expertise across all economies and regions of the world. With the power of the MERN stack behind us, we will provide a high-performance platform that brings professional assistance to the student's fingertips—regardless of how far the student is away from where they live or work.

Moreover, our vision is also reflected in how we are looking to address the social aspect of the student experience by incorporating technology as a means for destigmatization. Many students who need help suffer in silence due to the shame or fear they feel about attending a physical counseling office. By providing an anonymous and secure access point, Counsel-X will create a culture of resilience and openness. We want to help students not only address their immediate concerns but also to build long-term confidence and emotional intelligence that will enable them to succeed throughout their careers.

Counsel-X's long term mission extends beyond simply being an application; it is designed to be a true community partner to every educational institution in the world. Our goal is to establish ourselves as the “Gold Standard” of digital wellness platforms by combining our compassionate nature with our technical proficiency. Ultimately, we hope to create an ecosystem that is self-sustaining, where our customers do not feel like they are interacting with technology that is cold and complicated but rather warm and human. We see Counsel-X as a student's first place to turn when they feel stuck; thus, our customers would take a comprehensive “journey” through Counsel-X offering not only mental health advocacy and career planning but also transparency into their finances and a scheduled appointment in real-time, while also providing secure messaging will change the world one individual at a time by helping to create the next generation of well-informed, balanced, and successful professionals.

## 1.3 Related System Analysis

In order for Counsel-X to provide a unique and valuable service, an examination of similar digital solutions currently available in the marketplace is critical. The existing services provided to students are divided among global therapy, university-managed, and student-run peer-to-peer support systems, all of which have specific strengths, but also have significant gaps which Counsel-X hopes to fill.

Among the foremost global support systems is BetterHelp which connects users with licensed therapists via a subscription-based mobile app. While BetterHelp is a great option for anyone looking for clinical mental health therapy; it operates in such a way as to meet the needs of an adult audience. For students, this creates a “Context Gap”. A student may be seeking advice on how to select the right classes, or how to deal with the stress of taking exams; but general therapists may not have expertise in these specific areas. Additionally, BetterHelp has a very high-cost subscription-based model which most students cannot afford since they typically do not have the financial means to afford an ongoing subscription. Counsel-X solves this problem by

offering a “Pay for Each Session” payment model allowing students to only pay for the help that they need, along with specialized modules for academic and career planning that general therapy systems do not provide.

On the contrary, countless postsecondary institutes have developed Traditional University Portals. These are web pages set up by each respective institution for students to easily schedule appointments for services that are available to them, but typically they provide little help because they are usually out-of-date, often provide poor usability, and do not typically have any method of providing students with access to the institution’s personnel (i.e., the only way a student can connect/send an email to someone is via Email) therefore the only way for a student to get to know someone on campus is through Email; and if the person that they send an email to is busy, there is no other way for them to get any assistance. Counsel-X improves this situation by providing a modern and fast interface (including real-time displays for online/offline status) that instantly provides students with immediate access to a person with whom they can speak directly, via live chat, rather than delaying for an email to be returned that may take days before they get any assistance whatsoever.

Table 1.1: Comparative Analysis of Existing Systems

| Application Name   | Identified Weaknesses                                                                                                          | Proposed Improvements                                                                                                                              |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| BetterHelp         | <ul style="list-style-type: none"> <li>• Lacks academic-specific guidance.</li> <li>• Expensive subscription model.</li> </ul> | <ul style="list-style-type: none"> <li>• Dedicated modules for course/career planning.</li> <li>• Student-friendly per-session payment.</li> </ul> |
| University Portals | <ul style="list-style-type: none"> <li>• Outdated UI/UX.</li> <li>• Manual scheduling bottlenecks.</li> </ul>                  | <ul style="list-style-type: none"> <li>• Modern React-based responsive design.</li> <li>• Automated scheduling and reminders.</li> </ul>           |

## 1.4 Project Deliverables

A successful implementation of Counsel-X will lead to a number of tangible products, which will be divided into software modules and documentation products.

### List of Deliverables:

1. **Fully Functional Web Platform:** A responsive application built with React and Node.js, featuring real-time data synchronization.
2. **Student Module:** A dashboard allowing students to browse counselor profiles, book sessions, manage payments, and attend virtual meetings.
3. **Counselor Module:** An interface for professionals to manage their schedules, view upcoming appointments, and access student history (where permitted).

4. **Admin Module:** A central command center for user verification, platform monitoring, and financial reporting.
5. **Software Requirement Specification (SRS):** A comprehensive document detailing functional and non-functional requirements.
6. **Software Design Document (SDD):** Detailed architectural blueprints, including UML diagrams and Database Schema designs.
7. **Test Suite & Case Reports:** Documentation of unit, integration, and user-acceptance testing to ensure system reliability.
8. **Deployment Package:** A cloud-hosted version of the application with a configured MongoDB Atlas database.

## 1.5 System Limitations / Constraints

Despite its robust design, Counsel-X is subject to certain limitations:

- **Connectivity Dependency:** As a web-based platform, the system requires a stable internet connection for real-time counseling, which may be a barrier in low-bandwidth regions.
- **Hardware Constraints:** Users must have access to a device with a camera and microphone for video sessions.
- **Data Privacy Complexity:** Adhering to strict international data protection laws (like GDPR or HIPAA) for mental health records requires significant security overhead.
- **Scope Limitation:** The current version does not support emergency crisis intervention features, which require localized physical response teams.

## 1.6 Tools and Technologies

The selection of the technology stack was driven by the need for scalability, speed, and a modern development workflow. The following table provides a comprehensive list of the software tools and technologies utilized throughout the development lifecycle.

## 1.7 Relevance to Course Modules

### 1.7.1 Programming Fundamentals

The programming logic for Counsel-X follows many of the same programming concepts already covered in this module; for example, the use of object-oriented programming (OOP) to create reusable components in React, and the use of structured modelling in the backend through OOP to achieve this reusability. Another design principal that utilises OOP is the ability to separate the authentication logic from that of the session management, which allows for the code to be

Table 1.2: Tools and Technologies Used in the Project

| Category   | Name               | Version                | Purpose                 |
|------------|--------------------|------------------------|-------------------------|
| Tool       | Visual Studio Code | 1.56                   | Code Editor             |
| Tool       | MongoDB Compass    | 1.38                   | Database Management     |
| Tool       | Postman            | 10.15                  | API Testing             |
| Technology | React              | 18.3                   | Front-End Development   |
| Technology | Node.js            | 20.x (LTS)             | Server-Side Runtime     |
| Technology | MongoDB            | 6.0                    | NoSQL Database          |
| Technology | Socket.IO          | 4.7                    | Real-Time Communication |
| Technology | Stripe API         | API version 2024-06-07 | Payment Processing      |

modular and extendable. Lastly, the concepts of asynchronous programming (using Promises and Async/Await) will be used to manage API calls against the server in a non-blocking manner, and to ensure that the system is robust and reliable by implementing error and exception handling to manage unexpected runtime inputs.

## 1.7.2 Data Structures and Algorithms

The way that data is handled is critical to ensuring a proper schedule, and this required developing logic for validation of time slots as well as sorting counselors based on rating or availability. Both Array and Object data structures were highly utilized to manage real-time updates in Redux state and/or local component state, which allowed for an uninterrupted experience while performing these types of searches. Additional improvements were made to optimize the transmission of JSON-based data structures between clients and servers, through the use of optimized search algorithms, so that they could be filtered through rapidly when the user searches for help by filtering down their results based on many thousands of records of counselors will have a very low time complexity, assuring users have a scalable experience as the platform grows into the future.

## 1.7.3 Database Management Systems

Data stored with MongoDB was complex/non-relational data [e.g. user profile/session note]. To apply database design concepts like Normalization (as much as possible in NoSQL systems) and build relationships = references between the User and Appointment Collections to ensure data integrity and query performance. To expedite data retrieval, a user ID and a session created date have been indexed since they are always requested. Another key factor was atomicity to ensure a stable environment when dealing with sensitive financial transactions and reserving appointments with the MongoDB.

### 1.7.4 Software Engineering

Counsel-X used the Agile SDLC model to create Counsel-X; The iterative nature of this model helped in improving the application constantly. The UML (Use Case, Sequence, Class) diagrams were drawn prior to coding any of the code to visualize the system architecture. Several testing methodologies were used to ensure that all requirements stipulated in the Software Requirements Specification document were fulfilled through the black-box testing of the Login Module. Additionally, we utilized a variety of thorough requirement elicitation methods to identify gaps between stakeholder requirements and the technical specifications. In order to manage the code repositories for Counsel-X, we used version control systems (e.g., Git) to ensure that the design of all changes to the application can be documented and traced back to their original source.

### 1.7.5 Other Relevant Courses

The project was heavily influenced by Web Development; the main example of that influence was how to use REST APIs for the Front-end (FE) and Back-end (BE) to communicate with each other. Another example of how these principles were used included applying HCI principles while designing the dashboards, so that they would be easy to understand and use by students who may have high levels of stress. We also could apply what we learned in Corporate Cybersecurity courses by using JSON Web Tokens (JWT) and Bcrypt hashing algorithms to encrypt sensitive user data. Lastly, we could use our understanding of Cloud Computing and Networking to implement the application to a live server where the application can now be accessed at high speeds across the world.

## 1.8 Chapter Summary

The project was heavily influenced by Web Development; the main example of that influence was how to use REST APIs for the Front-end (FE) and Back-end (BE) to communicate with each other. Another example of how these principles were used included applying HCI principles while designing the dashboards, so that they would be easy to understand and use by students who may have high levels of stress. By using JSON Web Tokens (JWT) and Bcrypt hashing algorithms to secure sensitive user information, we were also able to apply what we learned in Corporate Cybersecurity courses. Finally, we were able to apply our Cloud Computing and Networking knowledge to deploy the application to a live server, allowing it to be accessed globally at high speeds.

## Chapter 2

### Problem Definition

#### 2.1 Problem Statement

There are also several systemic challenges facing traditional counseling services including a severe shortage of trained professionals and geographical limitations. Students who need guidance with their academic planning, career mapping and mental health, are very challenged to find quality services, especially those living in rural/under-served areas. Currently available digital solutions provide very basic appointment scheduling capabilities without any of the following features: real-time interaction, secure payment processing and centralized educational resources; as a result, students and academic institutions experience increased levels of stress and confusion. Additionally, due to the lack of adequate security measures in place to protect user privacy, confidential conversations are at risk of being accessed by others. Closing these gaps is both technically and socially meaningful; to this end, a comprehensive, scalable and secure platform will be needed so that all students can obtain vital support regardless of their location or time constraints.

#### 2.2 Proposed Solution Overview

Counsel-X is a digital platform to connect students with trained Counsellors. It provides students with more access to qualified Counsellors through a user-friendly digital infrastructure. Counsel-X is built on the MERN stack, which includes MongoDB, Express, React, and Node.js. The platform will improve the effectiveness of academic counseling by having a real-time counseling experience. The live chat feature is enabled by Socket.IO, payment processing through Stripe, and an automated resource library with educational and self-care resources. Counsel-X is designed to improve the ability for students, Counsellors, and administrators to work together seamlessly by having a role-based dashboard system. By automating scheduling and notifications, Counsel-X improves the operational efficiencies of the academic counseling system and ensures students receive no gaps in service through timely access to support. The overall mission of Counsel-X is to provide an innovative, scalable, fully-accessible, and highly-secure digital counseling service that meets the needs of today's educational environment.

#### 2.3 Objectives of the Proposed System

- **BO-1:** Build a robust web based counseling service using MERN stack to create a fast, scalable application.
- **BO-2:** Implement Real Time Communication tools using Socket.IO to improve student access to guidance services.
- **BO-3:** Incorporate the Stripe payment gateway to provide students with a secure, transparent financial transaction environment.

- **BO-4:** Create a secure method for protecting data privacy and confidential information through strong Role-Based Access Control (RBAC) mechanisms.
- **BO-5:** Automate the various elements involved in managing counseling sessions such as scheduling, emailing confirmation/notification reminders, etc. This will reduce human error in both appointment setting and confirmation/reminding users of their appointments.

## 2.4 Scope of the System

Counsel-X's full website application includes three main users: Students, Counsellors, and Administrators. Modules include secure user authentication, real-time messaging, financial transactions for booking sessions, and a centralized digital library of educational resources. The application is designed to work across multiple browsers so that users can access it on desktop or mobile devices. The system as defined does not provide for Physical Health Assessments or Offline Emergency Medical Services, it is designed solely for virtual counseling. The operational target market for the system is Education and Private Practices providing Counseling Services. Although the application provides the foundation for communication, the continued success of the counseling services offered through Counsel-X will be dependent upon the quality of service delivered by Verified Counsellors /Professionals who become Labeled Partners.

## 2.5 System Architecture and Modules

This software design uses a three-layered design; it features three layers: the Presentation Tier (created with React), the Logic Tier (created with Node.js and Express), and the Data Tier (created with MongoDB). With a 3 tier architecture each tier has its own clear responsibilities, enabling easy system maintenance and enabling all of the system to scale independently. Depending on the requirements of the system, the system is built to be a modular solution, with the capability to compartmentalize features, such as real time communications or payment processing, and use RESTful web services as a means of exchanging information rapidly and efficiently, and yet still provide a responsive user experience. The fact that the core business logic has been decoupled with the communication and database layer means that the platform is highly reliable and secure, and that user interaction can be processed in real-time, without affecting the stability or integrity of the data contained within the system.

### 2.5.1 Module 1: Profile Management

This module acts as the first step of every user, controlling the whole lifecycle of a user account, initially through registration (with secure login enabled with create token with JWT) and the management of these accounts during the auth process. Each user type has a Role that defines the level of authorisation that user will have to the various features of the application - they are defined as Student / Counsellor / Admin. The functionality of this module allows the user to manage their profile, update their credentials and view their own dashboard. This ensures that any sensitive information is only available to the intended user.

#### Functional Requirements:

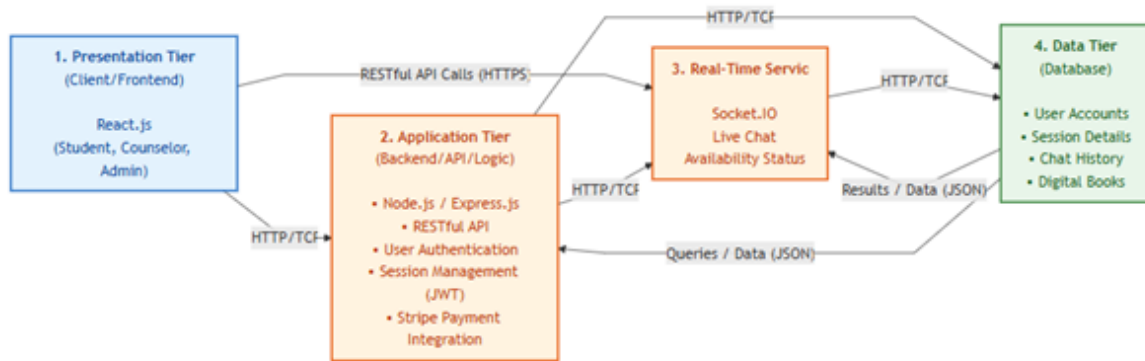


Figure 2.1: System Architecture Diagram

- FE-1: The system shall allow users to register using valid credentials and role selection.
- FE-2: The system shall authenticate users and provide role-based access to dashboards.

## 2.5.2 Module 2: Communication Module

A Communication Module is designed to facilitate all forms of interpersonal interactions across all platforms; this module will use Socket.IO to allow all students and Counsellors to have a low-latency real-time chat during an appointment. As a part of this communication module, webhook notifications will be sent to users regarding any appointment changes made by the system. Additionally, the communication module will maintain a secure log of all communications designed for accountability and to preserve the continuity of support offered.

### Functional Requirements:

- FE-1: The system shall enable secure, real-time messaging between students and Counsellors.
- FE-2: The system shall generate real-time push notifications for session updates.
- FE-3: The system shall maintain encrypted communication logs for history retrieval.

## 2.5.3 Module 3: Session and Booking Management Module

With this module you will get Counsel-X's primary functional benefits by managing the counseling workflow. This includes a scheduling engine where students can find Counsellors that fit their purchase courses and browse the counselor's calendar for available appointment times. Additionally, the module provides management of all session states associated with those booked sessions-ensuring that time slots are accurately reserved and guaranteeing that available time slots are actually available (i.e., cannot be double booked) by providing real-time availability updates.

### Functional Requirements:

- FE-1: The system shall allow students to search for Counsellors and view their real-time availability.

- FE-2: The system shall enable students to book counseling sessions for specific dates and times.
- FE-3: The system shall prevent overlapping appointments through synchronized scheduling logic.

#### 2.5.4 Module 4: Administrative and Resource Management Module

The purpose of this section is to provide administrators with everything they need to manage/enhance the platform. Administrators will utilize this section to validate qualifications of all new Counsellors, thereby ensuring that the Counsellors who interact with students are trained and qualified professionals. Administrators also manage their digital library through this section by uploading new or existing materials as well as updating/categorizing previously uploaded documents.

##### **Functional Requirements:**

- FE-1: The system shall allow administrators to verify counselor registrations and manage user account statuses.
- FE-2: The system shall enable the management of the digital library content, including PDFs and educational resources.
- FE-3: The system shall provide reporting tools to monitor system activity and resource usage.

#### 2.5.5 Module 5: Financial and Notification Services Module

The module includes integrations for essential external services that improve the overall reliability of the platform as well as provide additional revenue potential. The module can process secure financial transactions via Stripe's API, allowing for services such as purchasing a course and making payments for session-based services. In addition, the module automates the user engagement process via email and notification services that include reminding users of upcoming sessions, confirming payments made by users, and notifying users of system activity to ensure users have visibility into the operation of the system.

##### **Functional Requirements:**

- FE-1: The system shall integrate with the Stripe API to process secure financial transactions.
- FE-2: The system shall automate email notifications for session confirmations and schedule reminders.
- FE-3: The system shall track and summarize financial transaction history for users and administrators.

## 2.6 Assumptions and Dependencies

A number of assumptions and external dependencies underlie the design and successful deployment of Counsel-X:

- **Connectivity:** The assumption is that the user must be permanently and consistently connected to the internet to utilize the capabilities of video and real-time messaging.
- **Third-Party APIs:** This system is based on the assumption that Stripe (to process payments), Cloudinary and S3 (to store files), and MongoDB Atlas (to store data in the cloud) will be available 24 hours a day, 7 days a week.
- **Hardware Compatibility:** Assuming that, the users possess gadgets with an operational microphone and camera, the counseling-student process could be conducted through the video call feature provided in this system.
- **Email Services:** The third party SMTP services or email API providers (SendGRID or Mailgun) would have to be capable of communicating them reliably to the user.
- **User Literacy:** The assumption is that both the student and the Counsellors must be digitally literate enough to operate the modern web interface and manipulate digital files.
- **Credential Validity:** There is also an assumption that Counsellors possess appropriate and correct professional credential(s) and that Institutional Administrators will perform proper background checks on Counsellors prior to final approval of their credentials into this system.
- **Browser Support:** It is also assuming that the user has a modern web browser (Chrome, Firefox, and Safari) and that it can use ES6+, WebSockets and can render the CSS modernly.

## 2.7 Chapter Summary

This chapter explains the problem at hand and defines a conceptual solution that meets the outlined objectives, as well as addressing the challenges associated with geographical location and lack of real-time tools. The system's scope and architecture were also presented which show the structure of the system and include the major modules; Communication and Profile Management. Key assumptions and dependencies were also documented to clarify operational limitations; this chapter provides the basis for the problem and the direction for the detailed software design that will be developed in the next chapter.

## Chapter 3

### Requirement Analysis

#### 3.1 Introduction to Requirement Analysis

A formal specification of the system requirements for the Counsel-X project is stated in this chapter in a structured manner. The objective of this chapter is to detail what this system will actually do before any actual system design work can begin. The requirements laid out in this chapter will form a binding contract for implementation, validation and system testing.

In this chapter, we will present our analyses of these requirements by considering the elicitation techniques used to determine what needs to be done; who the stakeholders are; how we have modeled the use of the system through use cases; the functional, non-functional, and external interface requirements of the system; and that all requirements are written in such a way that they can be measured in an objective and reproducible manner based on standard requirements engineering principles.

#### 3.2 Requirement Elicitation Techniques

To specify the requirements for the Counsel-X project, we primarily used the use case modeling technique because there are several different users with different interactions with Counsel-X. We used the following methods to elicit and refine the system's requirements:

- **Stakeholder Interviews:** Briefly interviewed students to understand their issues in accessing counseling services while capturing user expectations and constraints.
- **Counsellor Consultations:** Discussed with professional counsellors how they manage their counseling sessions, therefore ensuring any system would meet the needs of the professional counseling industry.
- **Administrative Analysis:** Consulted administrative staff regarding user verification, content management needs, and reporting requirements.
- **Literature Review:** Reviewed similar systems and research contributions to identify functional gaps in existing online counselling solutions.
- **Feasibility Study:** Examined dataset requirements and technical constraints for real-time communication and secure payment processing.

#### 3.3 User Classes and Characteristics

This section identifies the different user categories interacting with Counsel-X. Each user class is defined along with their responsibilities and access privileges.

Table 3.1: User Classes and Characteristics

| User Class         | Description                                                                                             | Technical Level |
|--------------------|---------------------------------------------------------------------------------------------------------|-----------------|
| Administrator      | Manages system configuration, user accounts, approves registrations, and monitors activity reports.     | Advanced        |
| Counsellor         | Manages professional profiles, sets availability, conducts live chat, and evaluates scheduled sessions. | Intermediate    |
| Student (End User) | Accesses the system to book sessions, participate in live chat, and access the resource library.        | Basic           |
| External Systems   | API-based entities like Stripe or Clouinary interacting for payments and storage.                       | Technical       |

## 3.4 System Overview

A structured architecture can help develop modularly from the user interface layer through the logic processing layer to data storage. Counsel-X applies role-based access control and assists in implementing secure communications among modules within it and external services. In further detail, the boundaries of the system are:

- **User Management:** authentication and role-based access control.
- **Core Logic:** WebSocket real-time and session scheduling algorithms.
- **Financial Module:** safe payment processing to a third-party payment gateway.
- **Content Module:** Electronic library of learning materials (electronic resources).

## 3.5 Use Case Model

### 3.5.1 Use Case Diagrams

The functional requirements in the following use cases diagrams are depicted as the perspective of each user role.

#### 3.5.1.1 Login Use Case Diagram

The Login use case diagram: Both students and counsellors can log in. It demonstrates the way in which individuals use the system to create a new account or log into the old one. The following graphic includes key processes such as entering credentials, system verification and providing access to the appropriate dashboards. The login process ensures that only the appropriate individuals can access it and secures user data through the appropriate validation and authentication procedures.

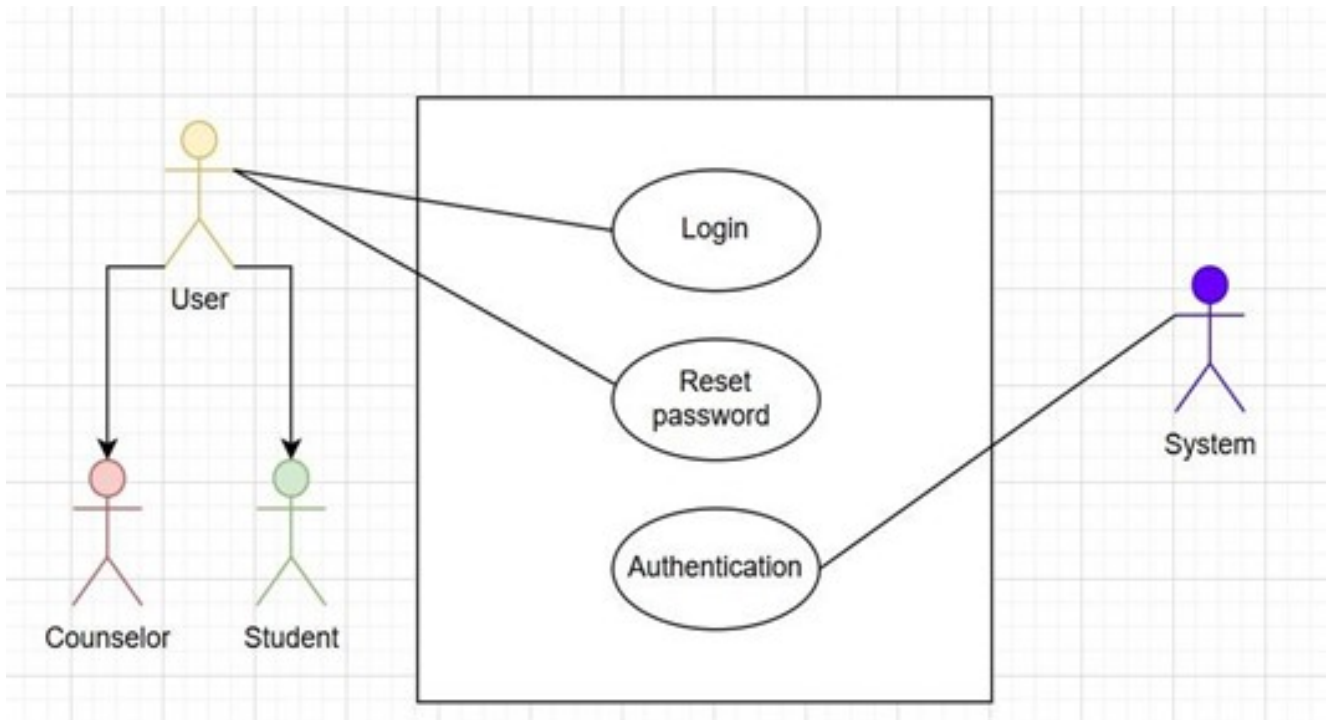


Figure 3.1: Login Use Case Diagram

#### 3.5.1.2 Student Use Case Diagram

The Student use case diagram, involves all the fundamental student interactions of Counsel-X, such as registering or logging in, scheduling counselling sessions, participating in live chat sessions, accessing the resource library and making secure payments. These roles assist students to get academic, career and mental-health counseling.

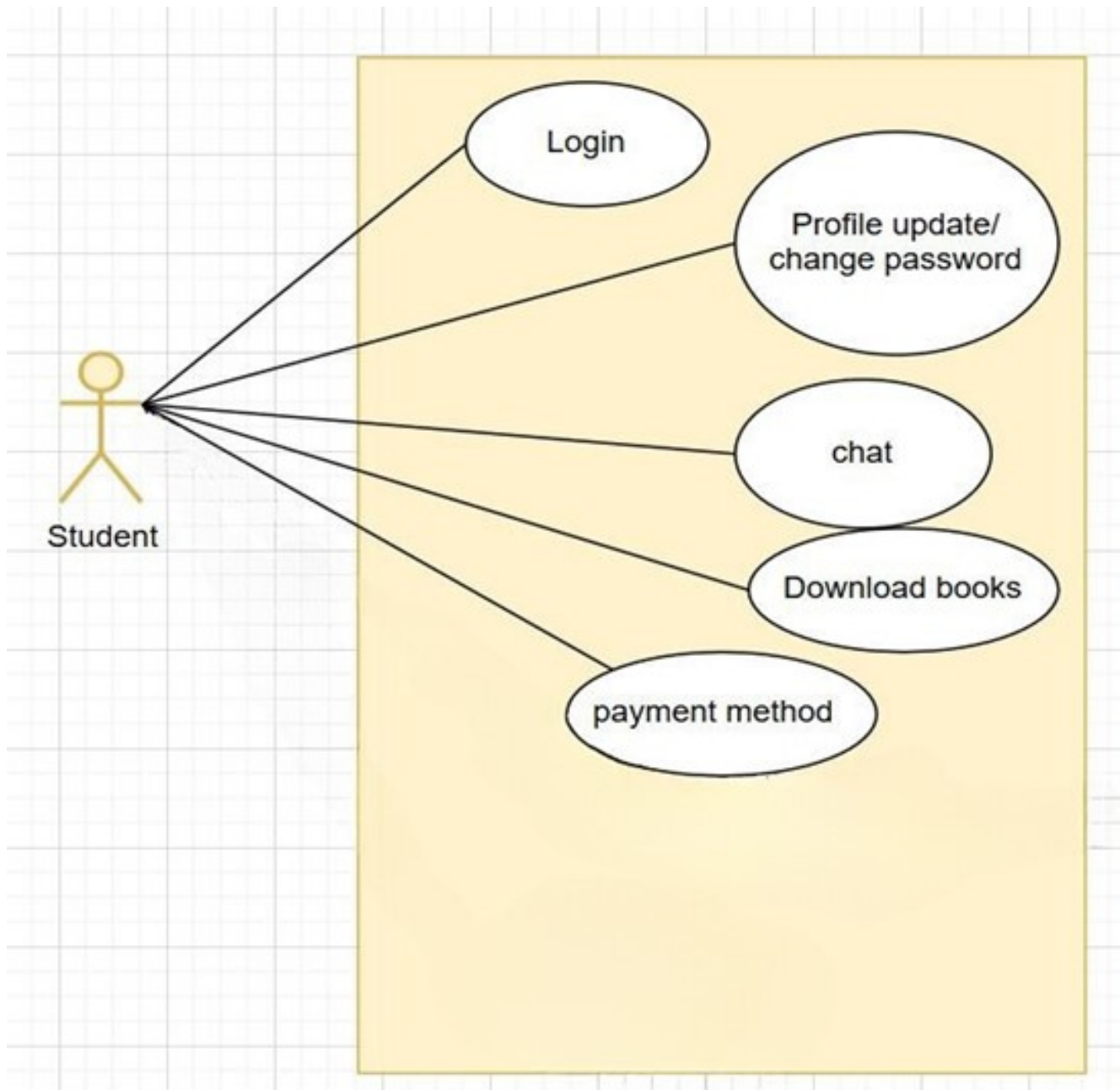


Figure 3.2: Student Use Case Diagram

### 3.5.1.3 Counsellor Use Case Diagram

The Counsellor use case diagram depicts the special activities that counsellors are involved with, which include keeping track of their availability, updating their professional profile, assessing the schedule appointment and live chat counselling. The features allow counsellors to support kids with the assistance they require and communicate with them.

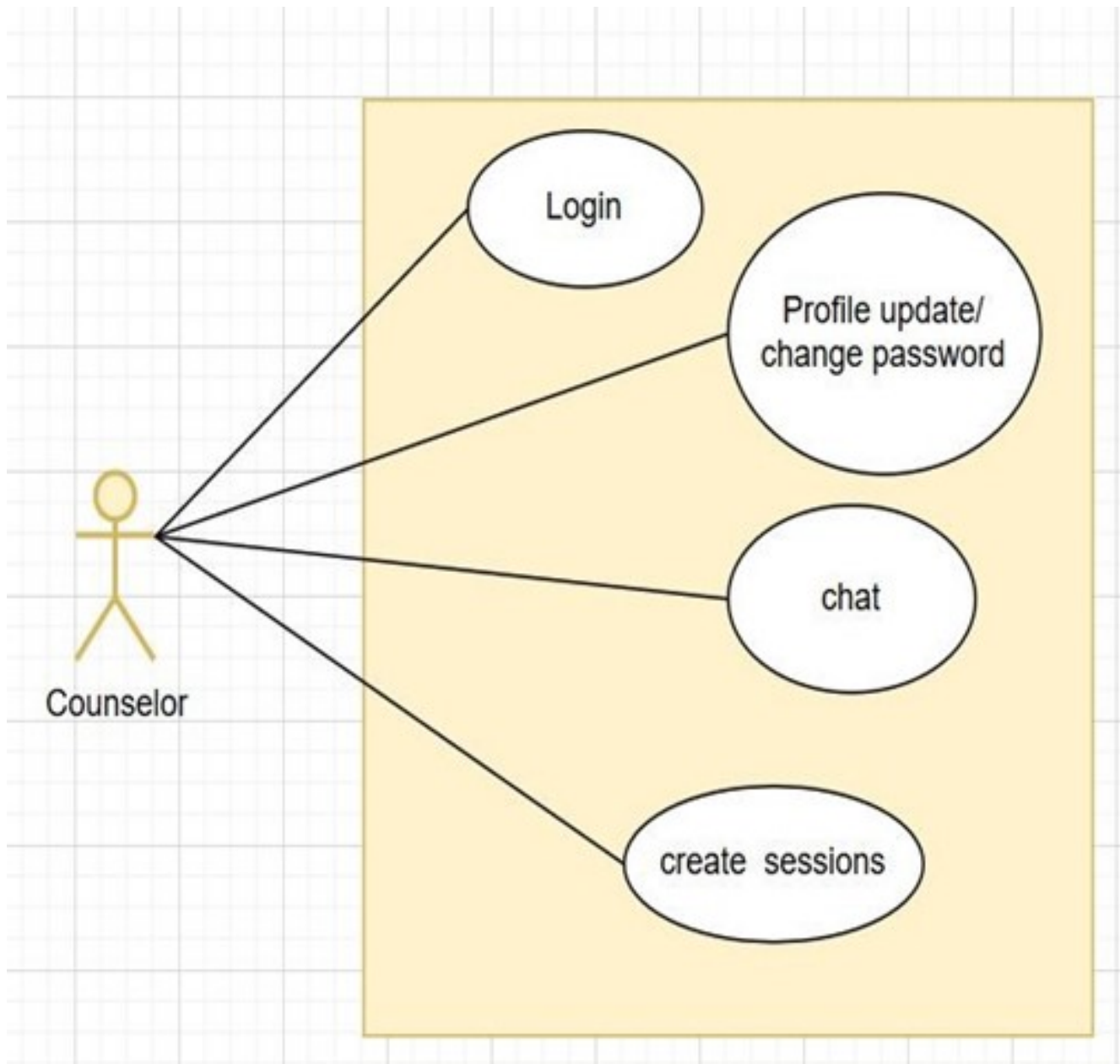


Figure 3.3: Counsellor Use Case Diagram

#### 3.5.1.4 Administrator Use Case Diagram

Administrator use case diagram illustrates top-level system processes such as the administration of user accounts, counsellor registration, the arrangement of content provided to the resource library, and the general system activity. Administrators are granted complete system privileges to provide the smooth running of the platform.

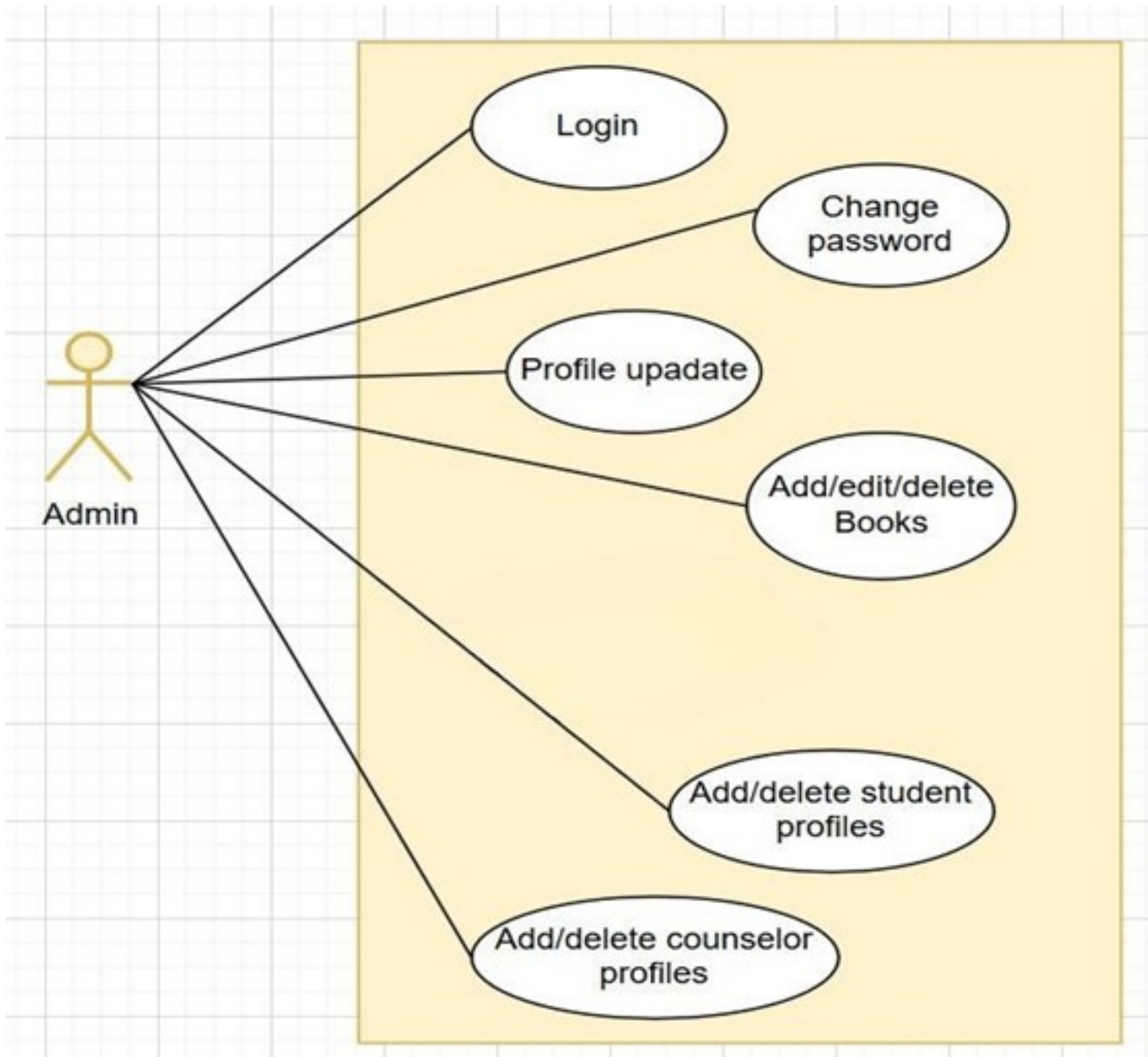


Figure 3.4: Administrator Use Case Diagram

## 3.5.2 Use Case Descriptions

### 3.5.2.1 UC-1: Login / Authentication

Table 3.2: UC-1: Login / Authentication

| Field            | Description                                                                                                                                                                                                                                                                              |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID      | UC-1                                                                                                                                                                                                                                                                                     |
| Use Case Name    | Login / Authentication                                                                                                                                                                                                                                                                   |
| Actors           | Student, Counselor, Admin, System                                                                                                                                                                                                                                                        |
| Description      | Users log into the system to access their profile and functionalities using secure credentials.                                                                                                                                                                                          |
| Trigger          | User opens the login page and enters email and password.                                                                                                                                                                                                                                 |
| Preconditions    | User must have an active registered account in the system.                                                                                                                                                                                                                               |
| Postconditions   | User is authenticated and redirected to their respective dashboard.                                                                                                                                                                                                                      |
| Normal Flow      | <ol style="list-style-type: none"> <li>1. User enters registered email and password.</li> <li>2. User clicks the "Login" button.</li> <li>3. System validates credentials against database.</li> <li>4. System generates JWT token.</li> <li>5. User redirected to dashboard.</li> </ol> |
| Alternative Flow | If credentials are invalid, system displays error message and prompts retry or reset.                                                                                                                                                                                                    |

### 3.5.2.2 UC-2: Manage Profile

Table 3.3: UC-2: Manage Profile

| Field            | Description                                                                                                                                                                                                |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID      | UC-2                                                                                                                                                                                                       |
| Use Case Name    | Manage Profile                                                                                                                                                                                             |
| Actors           | Student, Counselor, System                                                                                                                                                                                 |
| Description      | Allows users to view and update their personal or professional information.                                                                                                                                |
| Trigger          | User selects profile or settings from the dashboard.                                                                                                                                                       |
| Preconditions    | User is successfully logged in.                                                                                                                                                                            |
| Postconditions   | Personal information is updated in the database.                                                                                                                                                           |
| Normal Flow      | <ol style="list-style-type: none"> <li>1. Navigate to profile settings.</li> <li>2. Modify desired fields.</li> <li>3. Click "Save Changes".</li> <li>4. System validates and updates database.</li> </ol> |
| Alternative Flow | If validation fails, system shows error and requests valid input.                                                                                                                                          |

### 3.5.2.3 UC-3: Browse Counselors

Table 3.4: UC-3: Browse Counselors

| Field            | Description                                                                                                                                                                                                              |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID      | UC-3                                                                                                                                                                                                                     |
| Use Case Name    | Browse Counselors                                                                                                                                                                                                        |
| Actors           | Student, System                                                                                                                                                                                                          |
| Description      | Students can search and filter available counselors based on specialties.                                                                                                                                                |
| Trigger          | Student clicks on "Find a Counselor".                                                                                                                                                                                    |
| Preconditions    | Student is authenticated.                                                                                                                                                                                                |
| Postconditions   | A filtered list of counselors is displayed.                                                                                                                                                                              |
| Normal Flow      | <ol style="list-style-type: none"> <li>1. Enter search term/filter.</li> <li>2. System processes query.</li> <li>3. System fetches profiles from MongoDB.</li> <li>4. List is displayed with ratings and bio.</li> </ol> |
| Alternative Flow | If no match, system suggests broader filters.                                                                                                                                                                            |

### 3.5.2.4 UC-4: Schedule Session

Table 3.5: UC-4: Schedule Session

| Field            | Description                                                                                                                                                                                                |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID      | UC-4                                                                                                                                                                                                       |
| Use Case Name    | Schedule Session                                                                                                                                                                                           |
| Actors           | Student, Counselor, System                                                                                                                                                                                 |
| Description      | Student selects a specific time slot to book a counseling session.                                                                                                                                         |
| Trigger          | Student clicks on "Book Session".                                                                                                                                                                          |
| Preconditions    | Student has purchased a course or has credits.                                                                                                                                                             |
| Postconditions   | Appointment is reserved in both calendars.                                                                                                                                                                 |
| Normal Flow      | <ol style="list-style-type: none"> <li>1. View availability calendar.</li> <li>2. Select time slot.</li> <li>3. Confirm booking.</li> <li>4. System reserves slot and sends confirmation email.</li> </ol> |
| Alternative Flow | If slot is taken, system asks to choose another.                                                                                                                                                           |

### 3.5.2.5 UC-5: Live Chat Session

Table 3.6: UC-5: Live Chat Session

| Field            | Description                                                                                                                                                                                                |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID      | UC-5                                                                                                                                                                                                       |
| Use Case Name    | Live Chat Session                                                                                                                                                                                          |
| Actors           | Student, Counselor, Socket.io (System)                                                                                                                                                                     |
| Description      | Facilitates real-time text-based counseling during the scheduled window.                                                                                                                                   |
| Trigger          | Scheduled session time begins.                                                                                                                                                                             |
| Preconditions    | Valid session booked; current time within 45-minute window.                                                                                                                                                |
| Postconditions   | Session completed and chat history logged.                                                                                                                                                                 |
| Normal Flow      | <ol style="list-style-type: none"> <li>1. Join chat room.</li> <li>2. Socket.io establishes connection.</li> <li>3. Exchange real-time messages.</li> <li>4. Chat disabled when window expires.</li> </ol> |
| Alternative Flow | System attempts auto-reconnection on loss.                                                                                                                                                                 |

### 3.5.2.6 UC-6: Access Digital Library

Table 3.7: UC-6: Access Digital Library

| Field            | Description                                                                                                                                                                                                |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID      | UC-6                                                                                                                                                                                                       |
| Use Case Name    | Access Digital Library                                                                                                                                                                                     |
| Actors           | Student, Cloudinary (System)                                                                                                                                                                               |
| Description      | Students access educational materials, PDFs, and resources.                                                                                                                                                |
| Trigger          | Student navigates to "Digital Library".                                                                                                                                                                    |
| Preconditions    | Student is logged in.                                                                                                                                                                                      |
| Postconditions   | Selected resource is previewed or downloaded.                                                                                                                                                              |
| Normal Flow      | <ol style="list-style-type: none"> <li>1. Browse categories.</li> <li>2. Select specific document.</li> <li>3. System fetches asset from Cloudinary.</li> <li>4. Resource is opened/downloaded.</li> </ol> |
| Alternative Flow | If missing, system shows "Content Not Available".                                                                                                                                                          |

### 3.5.2.7 UC-7: Purchase Course

Table 3.8: UC-7: Purchase Course

| Field            | Description                                                                                                                                                                                                   |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID      | UC-7                                                                                                                                                                                                          |
| Use Case Name    | Purchase Course                                                                                                                                                                                               |
| Actors           | Student, Stripe API, System                                                                                                                                                                                   |
| Description      | Student pays for counseling packages via secure gateway.                                                                                                                                                      |
| Trigger          | Student clicks "Purchase".                                                                                                                                                                                    |
| Preconditions    | Student has valid payment method.                                                                                                                                                                             |
| Postconditions   | Payment confirmed; enrollment granted.                                                                                                                                                                        |
| Normal Flow      | <ol style="list-style-type: none"> <li>1. Proceed to checkout.</li> <li>2. Redirect to Stripe.</li> <li>3. Process payment within 5 seconds.</li> <li>4. System updates status and issues receipt.</li> </ol> |
| Alternative Flow | If fails, Stripe prompts for different card.                                                                                                                                                                  |

### 3.5.2.8 UC-8: Verify Counselor

Table 3.9: UC-8: Verify Counselor

| Field            | Description                                                                                                                                                                 |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID      | UC-8                                                                                                                                                                        |
| Use Case Name    | Verify Counselor                                                                                                                                                            |
| Actors           | Administrator, System                                                                                                                                                       |
| Description      | Admin reviews certificates and verifies counselor identity.                                                                                                                 |
| Trigger          | Counselor submits profile for review.                                                                                                                                       |
| Preconditions    | Admin is logged into management portal.                                                                                                                                     |
| Postconditions   | Counselor status changed to "Verified" or "Rejected".                                                                                                                       |
| Normal Flow      | <ol style="list-style-type: none"> <li>1. Access pending list.</li> <li>2. Review uploaded docs.</li> <li>3. Click "Approve".</li> <li>4. System updates status.</li> </ol> |
| Alternative Flow | Rejection with comments for re-upload.                                                                                                                                      |

### 3.5.2.9 UC-9: Manage User Accounts

Table 3.10: UC-9: Manage User Accounts

| Field            | Description                                                                                                                                                                                 |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID      | UC-9                                                                                                                                                                                        |
| Use Case Name    | Manage User Accounts                                                                                                                                                                        |
| Actors           | Administrator, System                                                                                                                                                                       |
| Description      | Admin performs management tasks such as account deactivation.                                                                                                                               |
| Trigger          | Routine management or policy violation report.                                                                                                                                              |
| Preconditions    | Admin has full privileges.                                                                                                                                                                  |
| Postconditions   | User account status is modified.                                                                                                                                                            |
| Normal Flow      | <ol style="list-style-type: none"> <li>1. Search for user.</li> <li>2. Select account.</li> <li>3. Choose action (Deactivate/Edit).</li> <li>4. System confirms and logs action.</li> </ol> |
| Alternative Flow | System prevents deactivation of last Admin.                                                                                                                                                 |

### 3.5.2.10 UC-10: Manage Library Content

Table 3.11: UC-10: Manage Library Content

| Field            | Description                                                                                                                                                                            |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID      | UC-10                                                                                                                                                                                  |
| Use Case Name    | Manage Library Content                                                                                                                                                                 |
| Actors           | Administrator, Cloudinary, System                                                                                                                                                      |
| Description      | Admin adds or removes resources from the platform.                                                                                                                                     |
| Trigger          | New content availability.                                                                                                                                                              |
| Preconditions    | Admin is logged in.                                                                                                                                                                    |
| Postconditions   | Resource library is updated.                                                                                                                                                           |
| Normal Flow      | <ol style="list-style-type: none"> <li>1. Select "Add Resource".</li> <li>2. Upload file.</li> <li>3. System sends to Cloudinary.</li> <li>4. System stores URL in MongoDB.</li> </ol> |
| Alternative Flow | Error notification if file exceeds size limit.                                                                                                                                         |

### 3.5.3 Event-Response Specifications

#### 3.5.3.1 EV-1: Login Attempt Response

Table 3.12: EV-1: Login Attempt Response

| Field            | Description                                                                                                                                                                                               |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID      | EV-1                                                                                                                                                                                                      |
| Use Case Name    | Login Attempt Response                                                                                                                                                                                    |
| Actors           | User, System                                                                                                                                                                                              |
| Description      | The system processes a login request to verify user identity.                                                                                                                                             |
| Trigger          | User clicks the "Submit" button on the login form.                                                                                                                                                        |
| Preconditions    | User has entered their email and password.                                                                                                                                                                |
| Postconditions   | User is either logged in or notified of failure.                                                                                                                                                          |
| Normal Flow      | <ol style="list-style-type: none"> <li>1. System hashes input password.</li> <li>2. System compares with database hash.</li> <li>3. System generates JWT token.</li> <li>4. Access is granted.</li> </ol> |
| Alternative Flow | If hash does not match, display "Invalid Credentials" and log failed attempt.                                                                                                                             |

### 3.5.3.2 EV-2: Payment Submission Response

Table 3.13: EV-2: Payment Submission Response

| Field            | Description                                                                                                                                                                                                                                |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID      | EV-2                                                                                                                                                                                                                                       |
| Use Case Name    | Payment Submission Response                                                                                                                                                                                                                |
| Actors           | Student, Stripe API, System                                                                                                                                                                                                                |
| Description      | System processes a payment event via the Stripe gateway.                                                                                                                                                                                   |
| Trigger          | Student confirms payment in the checkout window.                                                                                                                                                                                           |
| Preconditions    | Stripe API is responsive and card details are provided.                                                                                                                                                                                    |
| Postconditions   | Transaction is settled and enrollment is updated.                                                                                                                                                                                          |
| Normal Flow      | <ol style="list-style-type: none"> <li>1. System initiates secure handshake with Stripe.</li> <li>2. Stripe processes within 5 seconds.</li> <li>3. Success signal sent back to system.</li> <li>4. Status updated in database.</li> </ol> |
| Alternative Flow | If transaction is declined, notify user and prompt for alternative payment.                                                                                                                                                                |

## 3.6 Functional Requirements

Functional requirements define the core services and behaviors the system shall provide.

### 3.6.1 Module 1: Authentication and Access

Table 3.14: FR-01: User Login Functionality

|               |                                                                                                                                      |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Identifier    | FR-01                                                                                                                                |
| Title         | User Login Functionality                                                                                                             |
| Requirement   | The system shall authenticate Students, Counsellors, and Administrators using valid email and password credentials within 3 seconds. |
| Source        | Project Scope Document                                                                                                               |
| Rationale     | To ensure secure access and prevent unauthorized system entry.                                                                       |
| Business Rule | BR-01: Only registered users with verified email addresses can access the system.                                                    |
| Dependencies  | FR-02 (User Registration)                                                                                                            |
| Priority      | High                                                                                                                                 |

## 3.6.2 Module 2: Session & Communication Management

### 3.6.2.1 FR-2.1 Live Chat Session

Table 3.15: FR-03: Time-Bounded Chat Session

|                      |                                                                                                                                                         |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>    | FR-03                                                                                                                                                   |
| <b>Title</b>         | Time-Bounded Chat Session                                                                                                                               |
| <b>Requirement</b>   | The system shall enable chat functionality exclusively during the 45-minute scheduled session window, with automatic disabling upon session completion. |
| <b>Source</b>        | Final Project Report Section 2.2.4                                                                                                                      |
| <b>Rationale</b>     | To structure counselling sessions and ensure focused, time-bound interactions.                                                                          |
| <b>Business Rule</b> | BR-03: Chat input is disabled before session start and after session end time.                                                                          |
| <b>Dependencies</b>  | FR-02 (Session Booking)                                                                                                                                 |
| <b>Priority</b>      | High                                                                                                                                                    |

### 3.6.2.2 :FR-2.2 : Session Scheduling

Table 3.16: FR-02: Session Scheduling

|                      |                                                                                                                                           |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>    | FR-02                                                                                                                                     |
| <b>Title</b>         | Schedule counselling Session                                                                                                              |
| <b>Requirement</b>   | Students shall be able to book available counselling sessions by selecting counsellors and time slots that match their purchased courses. |
| <b>Source</b>        | Final Project Report Section 2.2.2.3                                                                                                      |
| <b>Rationale</b>     | Core functionality enabling student-counselor interaction scheduling.                                                                     |
| <b>Business Rule</b> | BR-02: Sessions can only be booked if the student has purchased the corresponding counselling course.                                     |
| <b>Dependencies</b>  | FR-04 (Course Purchase)                                                                                                                   |
| <b>Priority</b>      | High                                                                                                                                      |

### 3.6.3 Module 3: Profile and Resource Management

#### 3.6.3.1 FR-3.1 Profile Management

Table 3.17: FR-06: Counselor Profile Management

|                      |                                                                                                                                                |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>    | FR-06                                                                                                                                          |
| <b>Title</b>         | Counselor Profile Management                                                                                                                   |
| <b>Requirement</b>   | Counsellors shall be able to create and update professional profiles including qualifications, specialties, availability, and session pricing. |
| <b>Source</b>        | Final Project Report Section 2.2.3.1                                                                                                           |
| <b>Rationale</b>     | To enable counsellors to showcase expertise and manage service offerings.                                                                      |
| <b>Business Rule</b> | BR-06: Counselor profiles must be approved by administrators before becoming visible to students.                                              |
| <b>Dependencies</b>  | FR-01 (User Authentication)                                                                                                                    |
| <b>Priority</b>      | Medium                                                                                                                                         |

#### 3.6.3.2 Module 5: FR-3.2 Resource Management

Table 3.18: FR-07: Digital Library Management

|                      |                                                                                                                |
|----------------------|----------------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>    | FR-07                                                                                                          |
| <b>Title</b>         | Digital Library Management                                                                                     |
| <b>Requirement</b>   | Administrators shall be able to add, categorize, and manage digital books and resources in the system library. |
| <b>Source</b>        | Final Project Report Section 2.2.1.2                                                                           |
| <b>Rationale</b>     | To provide supplementary educational resources for platform users.                                             |
| <b>Business Rule</b> | BR-07: Library resources are accessible to all authenticated users based on their subscription level.          |
| <b>Dependencies</b>  | FR-05 (User Management)                                                                                        |
| <b>Priority</b>      | Low                                                                                                            |

### 3.6.4 Module 4: Payment and Administrative Management

#### 3.6.4.1 FR-4.1 Payment Processing

Table 3.19: FR-04: Payment Processing

|                      |                                                                                                                                    |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>Identifier</b>    | FR-04                                                                                                                              |
| <b>Title</b>         | Course Purchase and Payment                                                                                                        |
| <b>Requirement</b>   | Students shall be able to purchase counselling courses using Stripe payment gateway, with transaction completion within 5 seconds. |
| <b>Source</b>        | Final Project Report Section 2.2.2.2                                                                                               |
| <b>Rationale</b>     | To enable monetization and access control to counselling services.                                                                 |
| <b>Business Rule</b> | BR-04: Payment must be successfully processed before session booking is allowed.                                                   |
| <b>Dependencies</b>  | External (Stripe API)                                                                                                              |
| <b>Priority</b>      | High                                                                                                                               |

#### 3.6.4.2 FR-4.2: Administrative Management

Table 3.20: FR-05: Administrative User Management

|                      |                                                                                                             |
|----------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Field</b>         | <b>Description</b>                                                                                          |
| <b>Identifier</b>    | FR-05                                                                                                       |
| <b>Title</b>         | Administrative User Management                                                                              |
| <b>Requirement</b>   | Administrators shall be able to view all user accounts and deactivate accounts violating platform policies. |
| <b>Source</b>        | Final Project Report Section 2.2.1.1                                                                        |
| <b>Rationale</b>     | To maintain platform security and enforce community guidelines.                                             |
| <b>Business Rule</b> | BR-05: Only users with admin role can access user management functions.                                     |
| <b>Dependencies</b>  | FR-01 (User Authentication)                                                                                 |
| <b>Priority</b>      | Medium                                                                                                      |

## 3.7 Non-Functional Requirements

Quality attributes which describe how the system is going to be functioning are detailed through non-function requirements. Non-function requirements are also measurable and help determine reliability of the system as well as usability of the system, performance of the system, security of the system and maintainability of the system.

### 3.7.1 Performance Requirements

- **PER-1:** The main dashboard will load within three (3) seconds when in the normal operating state.
- **PER-2:** All real-time messages will be delivered with less than one (1) second latency for ninety-five percent (95%) of the time.

- **PER-3:** The system will have the capacity to support one hundred (100) concurrent users without any noticeable degradation in performance.
- **PER-4:** When a user searches for counsellors or resources the system will return results within two (2) seconds.

### 3.7.2 Security Requirements

- **SEC-1:** All sensitive information will be transmitted by using HTTPS protocol with TLS 1.2 or higher.
- **SEC-2:** Passwords will be hashed with Bcrypt before being stored in the database.
- **SEC-3:** Role-Based Access Control (RBAC) will be enforced rigidly preventing unauthorized access to the administrative controls.
- **SEC-4:** JWT Session Tokens will expire after 24 hours of inactivity in order to stop someone from using them to hijack a session.

### 3.7.3 Reliability Requirements

- **REL-1:** 99.9% uptime for the system must be maintained during the academic semesters.
- **REL-2:** No loss of data allowed due to system failure; the system must maintain transactional integrity.
- **REL-3:** Daily backups of the MongoDB database will be automated in order to recover data.

### 3.7.4 Usability Requirements

- **USE-1:** Users must be able to complete core tasks (e.g., book a session) with three clicks or less.
- **USE-2:** Users must be able to see real-time status of their assigned counsellors.
- **USE-3:** The system will comply with WCAG 2.1AA accessibility standards for providing users with disabilities the same access as other users.
- **USE-4:** The application will fully scale to mobile, tablet, and desktop devices.

### 3.7.5 Scalability Requirements

- **SCA-1:** The system's architecture must allow for the horizontal scaling of the backend services to accommodate ever-increasing user traffic.
- **SCA-2:** The database design must support high volume of both session logs and library resources.
- **SCA-3:** Additional third-party API integrations must not require any significant structural changes.

### 3.7.6 Maintainability Requirements

- **MAIN-1:** The system shall follow a modular MERN architecture to ensure ease of updates and debugging.
- **MAIN-2:** The source code shall be documented clearly with standardized naming conventions.
- **MAIN-3:** The system shall allow for the integration of new features or modules without affecting the stability of existing functionalities.

## 3.8 External Interface Requirements

This section defines how the system interacts with external entities including users, hardware devices, software systems, and communication networks.

### 3.8.1 User Interface Requirements

- **UI-1:** The system shall provide a graphical user interface based on modern web standards.
- **UI-2:** Standard font sizes shall ensure readability across different resolutions (12-14)pt.
- **UI-3:** The layout shall be fully responsive across mobile, tablet, and desktop devices.
- **UI-4:** Navigation shall remain consistent across all platform modules.

### 3.8.2 Hardware Interface Requirements

- **HI-1:** The system shall operate on standard computing devices (PCs, Laptops, Tablets).
- **HI-2:** For mobile access, the platform shall support standard Android and iOS mobile browsers.
- **HI-3:** Minimum hardware requirements include 4GB RAM and a modern processor for smooth real-time chat operation.

### 3.8.3 Software Interface Requirements

- **SI-1:** The system shall interact with a MongoDB non-relational database for storage.
- **SI-2:** Third-party APIs (Stripe, Cloudinary) shall be securely integrated.
- **SI-3:** Payment gateways shall use secure HTTPS callbacks for transaction verification.
- **SI-4:** Client-side logic shall be compatible with modern browsers (Chrome, Firefox, Safari).

### 3.8.4 Communication Interface Requirements

- **CI-1:** The system shall use HTTPS protocol with SSL/TLS encryption for all data communication.
- **CI-2:** RESTful APIs using JSON format shall be used for primary client-server communication.
- **CI-3:** Real-time communication shall use WebSockets protocol via Socket.io for low latency chat.
- **CI-4:** Automated email services shall use secure SMTP/API protocols for notifications.

### 3.8.5 Database Interface Requirements

- **DI-1:** The system shall maintain a persistent connection to MongoDB Atlas for cloud storage.
- **DI-2:** Database operations shall follow Mongoose ODM schemas to ensure data consistency.
- **DI-3:** Data retrieval and storage shall utilize JSON objects for efficient processing in the Node.js environment.

### 3.8.6 API Interface Requirements

- **AI-1:** Stripe API: Integration for processing secure payments and managing customer billing sessions.
- **AI-2:** Cloudinary API: Integration for uploading, storing, and serving profile images and library PDF resources.
- **AI-3:** Socket.io Interface: Handling events for real-time bidirectional message broadcasting during counseling.

## 3.9 Assumptions and Dependencies

- **User Literacy:** Users possess basic digital literacy and access to a modern web browser (Chrome, Firefox, Safari).
- **Cloud Services:** Continuous availability and reliability of external cloud services, specifically Stripe (Payments), Cloudinary (Media), and MongoDB Atlas (Database).
- **Connectivity:** Stable internet connectivity with a minimum bandwidth of 1-2 Mbps required for low-latency real-time functionality.
- **Credentials:** It is assumed that counselors will provide valid professional documentation and certificates for verification by administrators.

- **Data Integrity:** Users are assumed to provide accurate information during registration and profile management.
- **SMTP Infrastructure:** The system depends on a functional email delivery service (SMTP or API-based) for notifications and password recovery.
- **Security Updates:** Dependency on periodic updates of underlying libraries (Node.js, Express, React) to ensure system security and patch vulnerabilities.
- **Responsive Access:** Assumption that the majority of mobile users access the platform through devices that support modern CSS standards and ES6 JavaScript.

## 3.10 Requirement Traceability Matrix

Table 3.21: Requirement Traceability Matrix

| Objective    | Use Case         | Functional Requirement |
|--------------|------------------|------------------------|
| Secure Entry | UC-1             | FR-01                  |
| Guidance     | UC-3, UC-4, UC-5 | FR-02, FR-03           |
| Financials   | UC-7             | FR-04                  |
| Support      | UC-6             | FR-07                  |
| Management   | UC-8, UC-9       | FR-05, FR-06           |

## Chapter Summary

In this chapter, we presented a structured and comprehensive requirements analysis of the Counsel-X platform. Through several elicitation methods, including stakeholder interviews and stakeholder consultations, we identified the most critical needs of students, counsellors and administrators. The use case modeling provided clear functional pathways for meeting these identified needs, including secure access (authentication), real-time communication (messaging) and the ability to manage resources.

We also established several key non-functional measures of system performance, security and scalability in order to ensure that Counsel-X remains a reliable solution during periods of peak academic activity. The external interface requirements were established to define how the internal logic of the MERN stack will interact with third-party services like Stripe and Socket.io. Overall, the requirements in this chapter will serve as the formal basis for the architectural design and system implementation that will be described in detail in the following chapters by ensuring that every feature developed can be traced back to either a specific user need or a system objective.

## Chapter 4 Software Design

### 4.1 Introduction to Software Design

This chapter presents the detailed software design of Counsel-X, an online student counselling system. It translates the functional and non-functional requirements identified in Chapter 3 into structured architectural, behavioral, and data models. The purpose of this chapter is to provide a clear technical blueprint for system implementation, ensuring maintainability, scalability, and modular development.

The design process focuses on creating a seamless interaction between the React.js frontend, the Node.js/Express backend, and external integrations like Socket.io for real-time chat and Stripe for financial transactions.

### 4.2 Design Methodology and Software Process Model

#### 4.2.1 Design Methodology

The project follows an Object-Oriented Design (OOD) approach combined with a Layered Architecture (Model-View-Controller) and Service-Oriented principles.

- **Justification:** Counsel-X requires distinct separation between user interface logic (View), business rules like session timing (Controller), and data persistence (Model). This ensures that a change in the UI (React) does not break the backend logic. Furthermore, the layered approach allows for independent testing of the communication engine and payment services without affecting the core user management modules.
- **Architectural Layers:**
  - **Presentation Layer (View):** Developed using React.js, this layer is responsible for rendering the UI and capturing user inputs. It communicates with the backend via RESTful APIs and maintains local state for immediate user feedback.
  - **Application Layer (Controller):** Implemented using Express.js, this layer orchestrates the flow of data between the UI and the domain logic. It handles request routing, input validation, and session management.
  - **Domain Layer (Model):** This layer contains the core business logic, such as the algorithm for calculating the 45-minute chat window and the logic for verifying course subscriptions before allowing a booking.
  - **Infrastructure Layer:** Manages communication with external tools like the database, Stripe for payments, Cloudinary for file storage, and Socket.io for real-time messaging.
- **Design Principles:**

- **SOLID:** Includes Single Responsibility (e.g., PaymentService), Open/Closed (extending payment methods without modifying core logic), and Interface Segregation to prevent bloated dependencies.
- **Open/Closed:** The system is designed to be open for extension (e.g., adding a new payment method) but closed for modification of core booking logic.
- **Interface Segregation:** Clients only depend on the API methods they actually use, preventing bloated interfaces.
- **DRY (Don't Repeat Yourself):** Reusable React hooks are used for fetching data, and common backend middlewares (like isAdmin or isCounsellor) are shared across routes to reduce code redundancy.
- **Separation of Concerns:** Real-time communication logic is completely isolated from static resource management. This allows the Socket.io server to be scaled independently of the main API server if traffic increases.
- **Loose Coupling:** The frontend and backend communicate via standardized JSON formats, allowing either side to be refactored or swapped with minimal impact on the other.

#### 4.2.2 Software Process Model

The project follows the Agile Incremental Model to ensure systematic development and flexibility.

- **Selection Justification:** This model was selected because it allows the team to deliver core functionalities (like Authentication and Profile Management) in early stages while leaving complex, high-risk modules (like Stripe integration and Real-time Chat) for later increments. This reduces overall project risk and allows for early stakeholder feedback.
- **Iteration Cycles:** The development is divided into 2-week sprints. Each sprint follows a cycle of requirement refinement, design, coding, and review. This ensures that a functional increment of the software is produced at the end of every two weeks, allowing the platform to grow modularly.
- **Testing and Validation:** Testing is integrated into every phase of the cycle. Unit tests are performed on backend APIs, while functional testing validates the 45-minute session logic. Validation is performed at the end of each increment through sprint reviews, ensuring the system meets the user requirements before proceeding to the next phase.

### 4.3 System Overview

Counsel-X acts as a mediator between students seeking guidance and professional counsellors. The high-level interaction is captured in the Context Diagram.

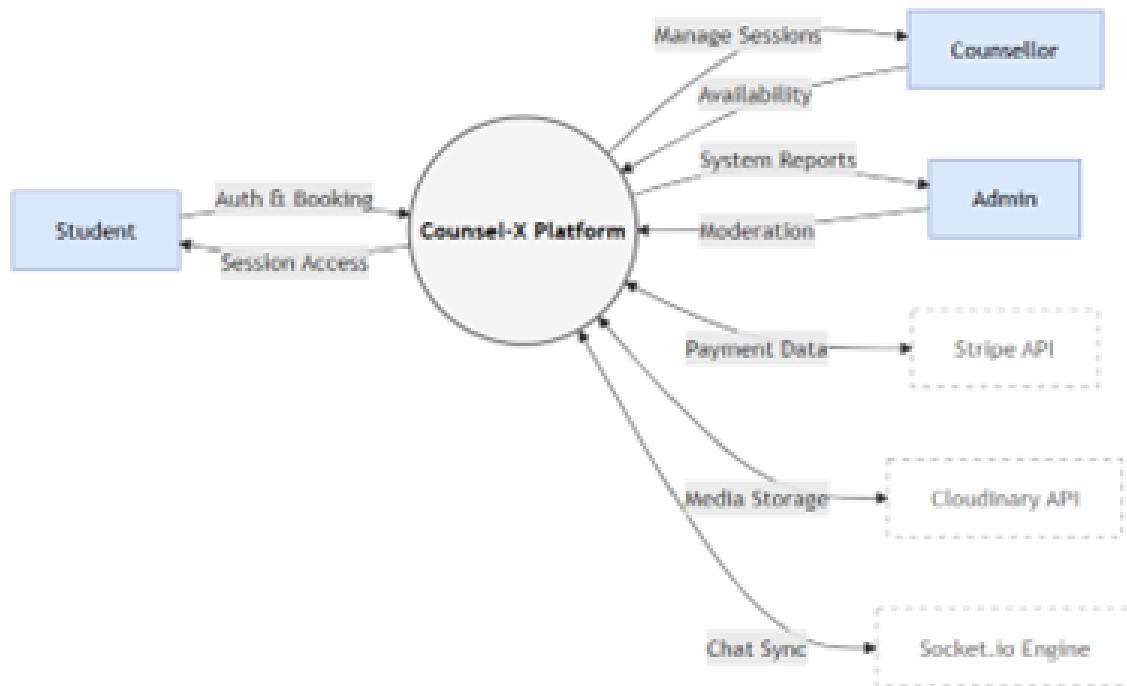


Figure 4.1: Context Diagram of Counsel-X

#### Explanation:

The system overview highlights the interaction between Students, Counsellors, and Administrators with the central Counsel-X platform. Students initiate authentication and booking while receiving session access, whereas Counsellors manage their availability and clinical sessions through the system. External services like Stripe, Cloudinary, and Socket.io are integrated to handle financial transactions, media storage, and real-time message synchronization, ensuring a cohesive and secure functional boundary for all stakeholders.

## 4.4 Architectural Design

The architecture of Counsel-X is structured into four distinct layers to ensure high availability, security, and performance. Each layer is decoupled, allowing for easier maintenance and future technological upgrades.

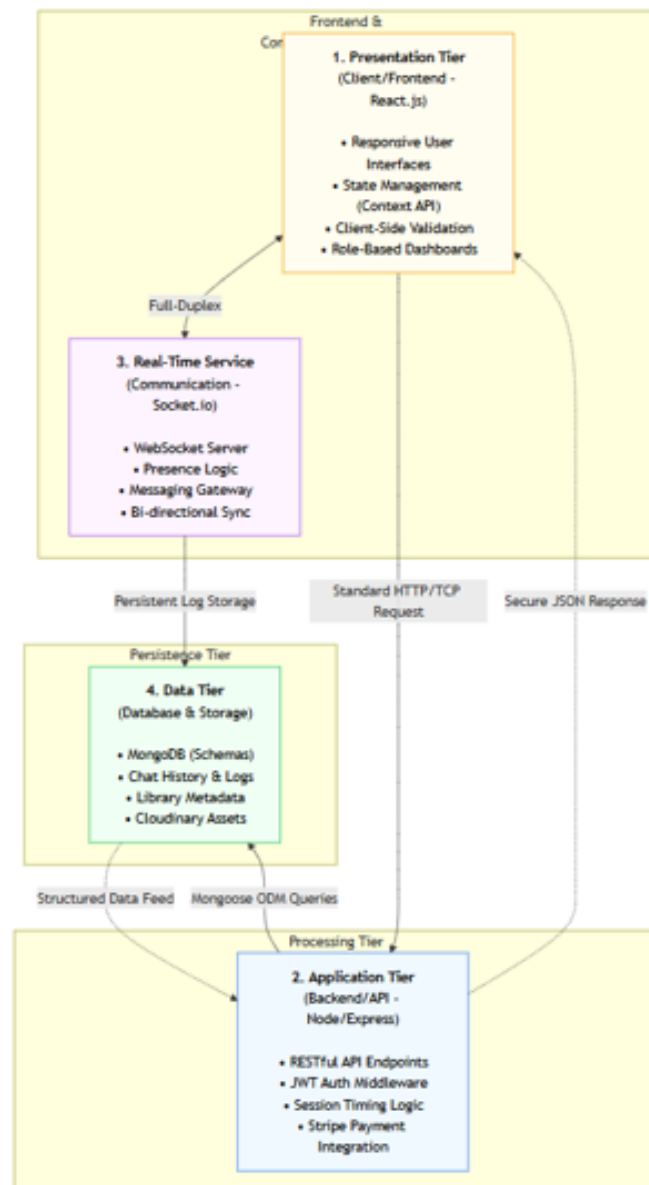


Figure 4.2: System Architecture Diagram

## 1. Presentation Layer (Frontend):

- **Technology:** Developed using React.js with Tailwind CSS for high-performance styling and responsiveness.
- **Functionality:** It manages complex user states using React Hooks (useState, useEffect, useContext) and ensures that data fetching is handled asynchronously via Axios. This layer is responsible for client-side routing, input sanitization, and providing a seamless interface for real-time chat interactions.

## 2. API/Gateway Layer:

- **Technology:** Node.js environment utilizing Express.js routing.

- **Functionality:** Acts as the entry point for all frontend requests. It handles security protocols such as Cross-Origin Resource Sharing (CORS) and implements JWT-based authentication middleware to protect sensitive routes. It also manages the initial handshake for WebSocket (Socket.io) connections, ensuring only authenticated users can join real-time communication rooms.

### 3. Business Logic Layer (Backend):

- **Technology:** Node.js with a modular service-oriented structure.
- **Functionality:** Known as the "brains" of the platform, this layer processes core business rules. Key modules include:
  - **Booking Service:** Automates schedule matching and prevents overlapping appointments.
  - **Session Manager:** Enforces the 45-minute chat window through server-side timers.
  - **Payment Logic:** Validates course purchases via the Stripe API before granting session access.
  - **Content Manager:** Handles the categorization and delivery of resources from the digital library.

### 4. Data Layer:

- **Technology:** A relational database (e.g., MySQL or PostgreSQL) for structured data and Cloudinary for media assets.
- **Functionality:** Stores persistent transactional data including user profiles, encrypted password hashes, payment history, and session logs. Unstructured data, such as counsellor profile pictures and PDF books from the resource library, are managed through Cloudinary to ensure low-latency media delivery through its CDN.

## 4.5 Design Models

Design models provide a behavioral and structural view of the system. These models help in understanding how different components of Counsel-X interact and how the control flow moves from one state to another during user operations.

### 4.5.1 Activity Diagrams

Activity diagrams are used to model the workflow of a system. They capture the dynamic behavior of Counsel-X by showing the sequence of activities and the conditional logic involved in core modules like authentication and session booking.

#### 4.5.1.1 Module 1: User Management & Authentication

The authentication process begins when a user navigates to the login interface and submits their credentials, specifically their registered email and encrypted password. Upon submission, the

system initiates a backend verification process where the provided information is compared against the records stored in the primary database. If the credentials match a valid account, the system generates a JSON Web Token (JWT) to establish a secure session and automatically redirects the user to their role-specific dashboard (Student, Counsellor, or Administrator). In the event of a validation failure, such as an incorrect password or non-existent email, the system denies access, displays an "Access Denied" error message, and provides the user with an option to initiate a password reset procedure to regain account access.

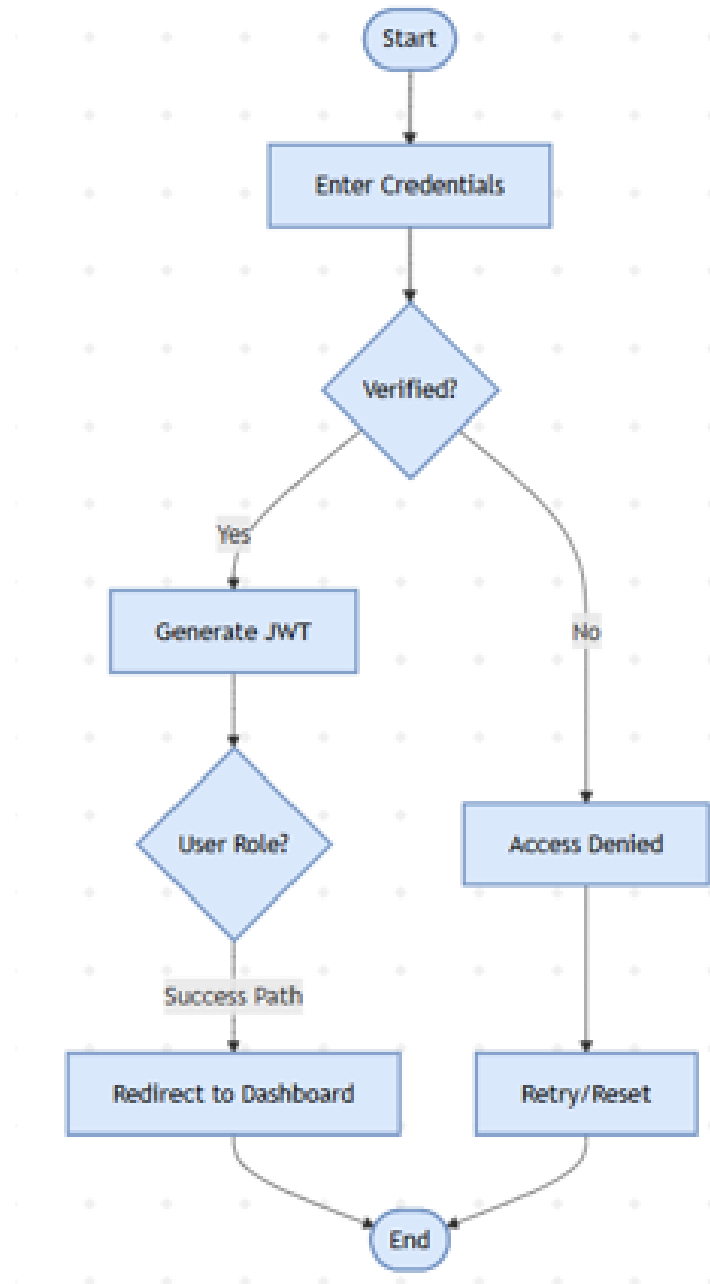


Figure 4.3: Activity Diagram – User Module

Explanation:

The flow begins with the user entering their email and password. The system performs a backend check; if the credentials are valid, a signed JSON Web Token (JWT) is generated. The system then evaluates the user's role and routes them to the appropriate dashboard. If verification fails, access is blocked, and the user is prompted with an "Access Denied" message and a password reset option.

#### 4.5.1.2 Module 2: Session Booking & Real-Time Chat

The core functional module governs the transition from administrative scheduling to active counselling. The process begins when a student views a counsellor's available schedule and selects a preferred time slot. Before confirmation, the system performs an automated check of the student's payment status via the financial gateway. Upon successful verification, the booking is recorded, and the system triggers a notification to the counsellor. During the scheduled appointment, the system monitors the active time window ( $T_{start}$  to  $T_{end}$ ). If the student or counsellor joins within the authorized 45-minute period, the system initializes a WebSocket connection via Socket.io to enable real-time messaging. To ensure strict adherence to the professional schedule, the system automatically terminates the communication link and disables the chat interface at  $(T_{start} \text{ to } T_{end}) + 1$  minute, effectively closing the session.

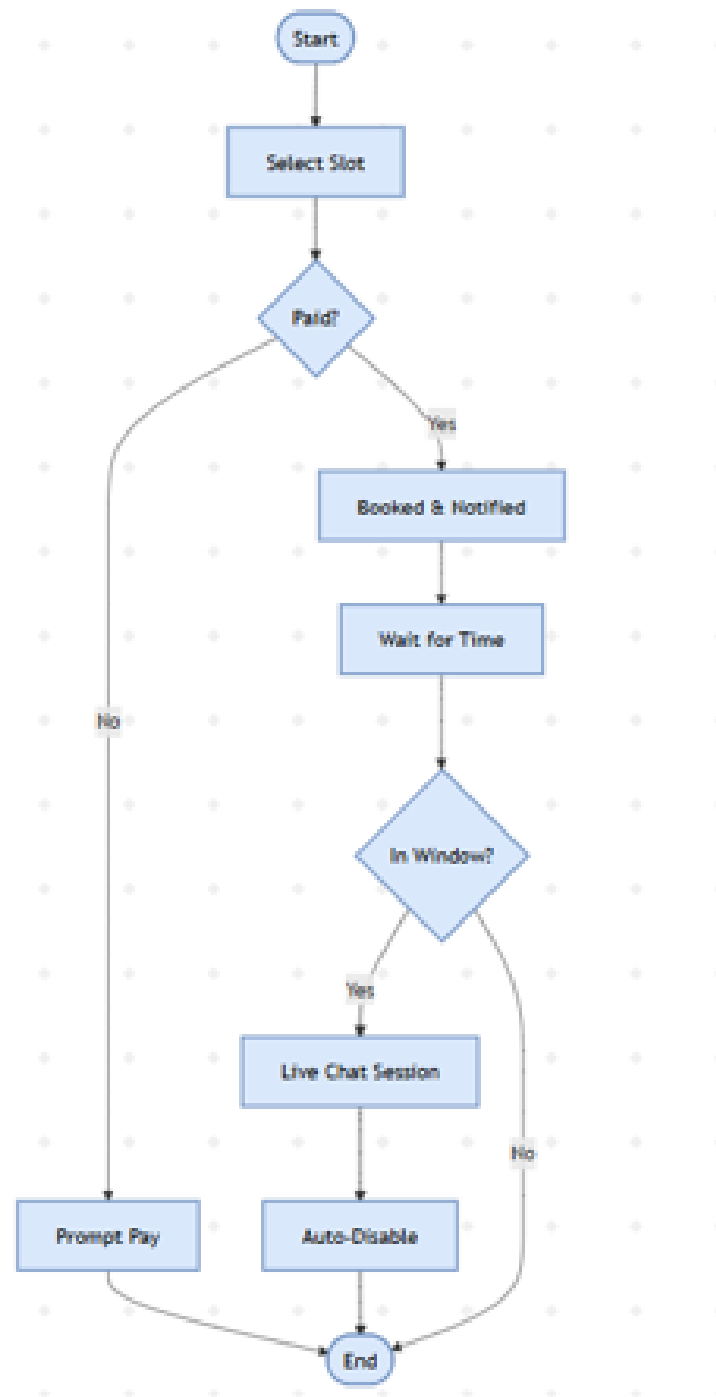


Figure 4.4: Activity Diagram – Core Functional Module

### Explanation:

The student initiates the process by selecting a time slot. The system checks if the student has paid for the course/package. If unpaid, it prompts for payment (Error path). Upon successful verification, the session is booked (Success path). At the scheduled time, the system checks if the user is within the 45-minute "window". If yes, real-time chat is enabled via Socket.io. Once the time expires, the input is auto-disabled to maintain professional boundaries.

#### 4.5.1.3 Module 3: Admin Module

This diagram covers monitoring, user control, and data management operations performed by the Administrator.

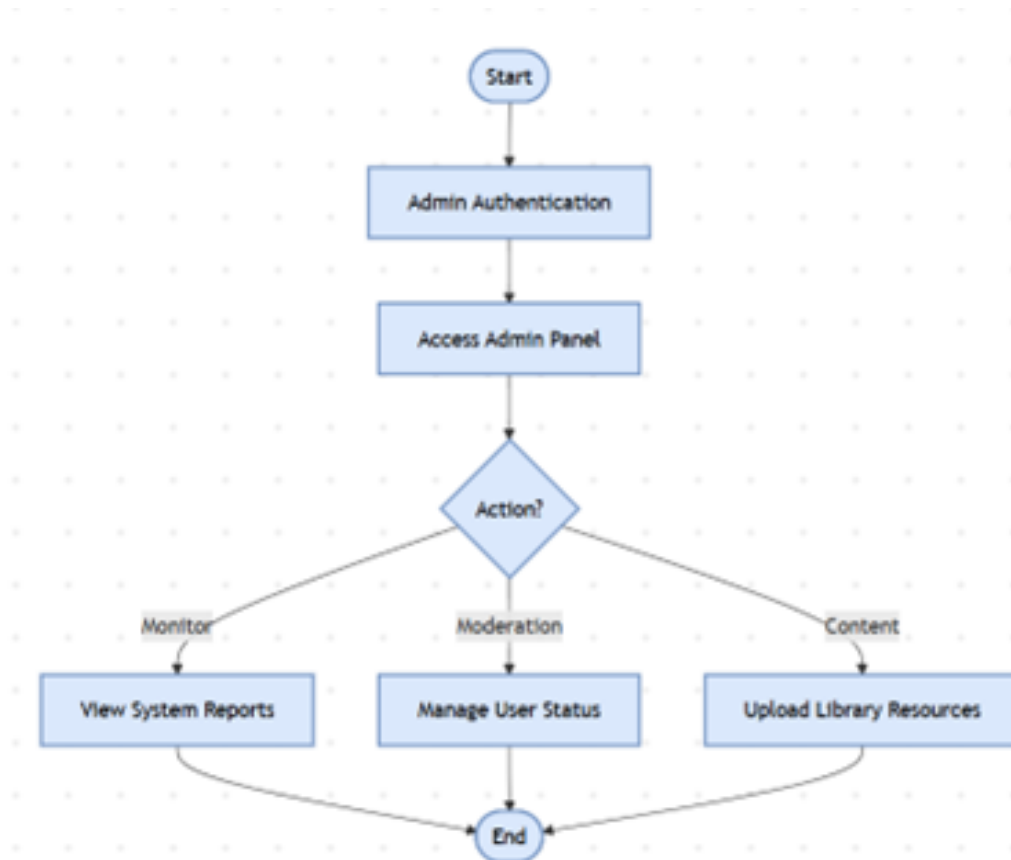


Figure 4.5: Activity Diagram – Admin Operations

#### Explanation:

The Admin Module activity diagram depicts the administrative workflow, beginning with secure authentication to grant access to the centralized dashboard. Administrators can navigate through key operational paths, including real-time system monitoring, user account moderation to ensure platform safety, and digital library management for educational resources. This structured flow ensures that high-level oversight and content delivery are managed efficiently through a verified and controlled interface.

#### 4.5.2 Class Diagram

The class diagram represents the static structure of Counsel-X. It defines the blueprints (classes) for objects such as Users, Students, and Sessions, detailing their attributes, methods, and inheritance relationships.

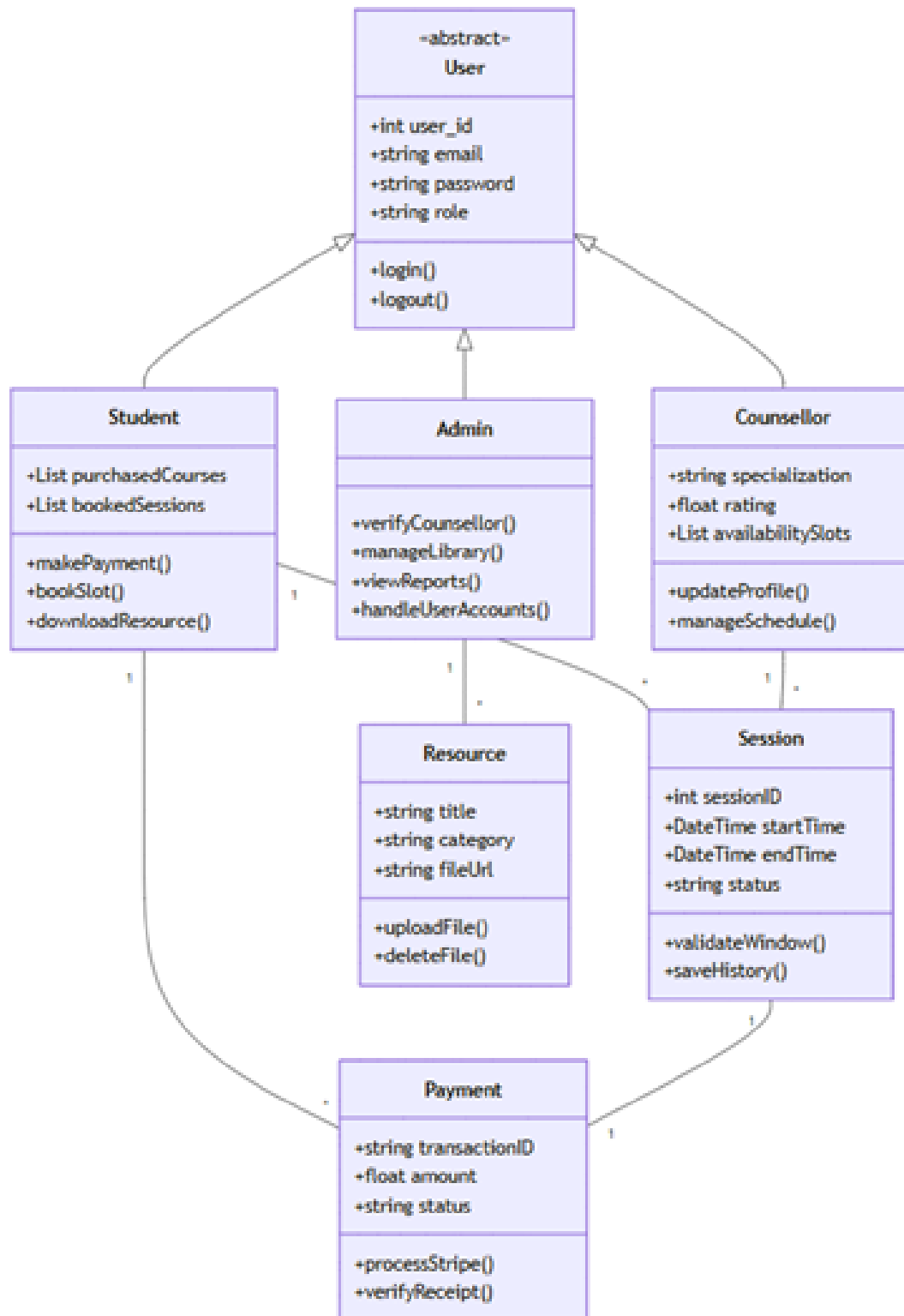


Figure 4.6: Class Diagram for Counsel-X

Explanation:

The Class Diagram for Counsel-X follows an object-oriented approach, centered around an abstract 'User' class that defines shared authentication logic for Student, Counsellor, and Admin roles. Functional relationships are strictly governed by the Session and Payment classes, ensuring that clinical counseling sessions are authorized only after a transaction is verified through the Stripe gateway. Furthermore, the Resource class provides a modular structure for managing the digital library metadata, allowing the Admin to control content that is seamlessly delivered to Students. This design ensures that financial processing, scheduling, and resource management interact through clearly defined architectural boundaries for maximum scalability.

### 4.5.3 Sequence Diagrams

#### 4.5.3.1 Sequence Diagram of Counselor

This diagram details the counsellor's operational workflow, focusing on profile management and the initiation of live sessions. It ensures that the counsellor's interface communicates correctly with the server-side session logic.

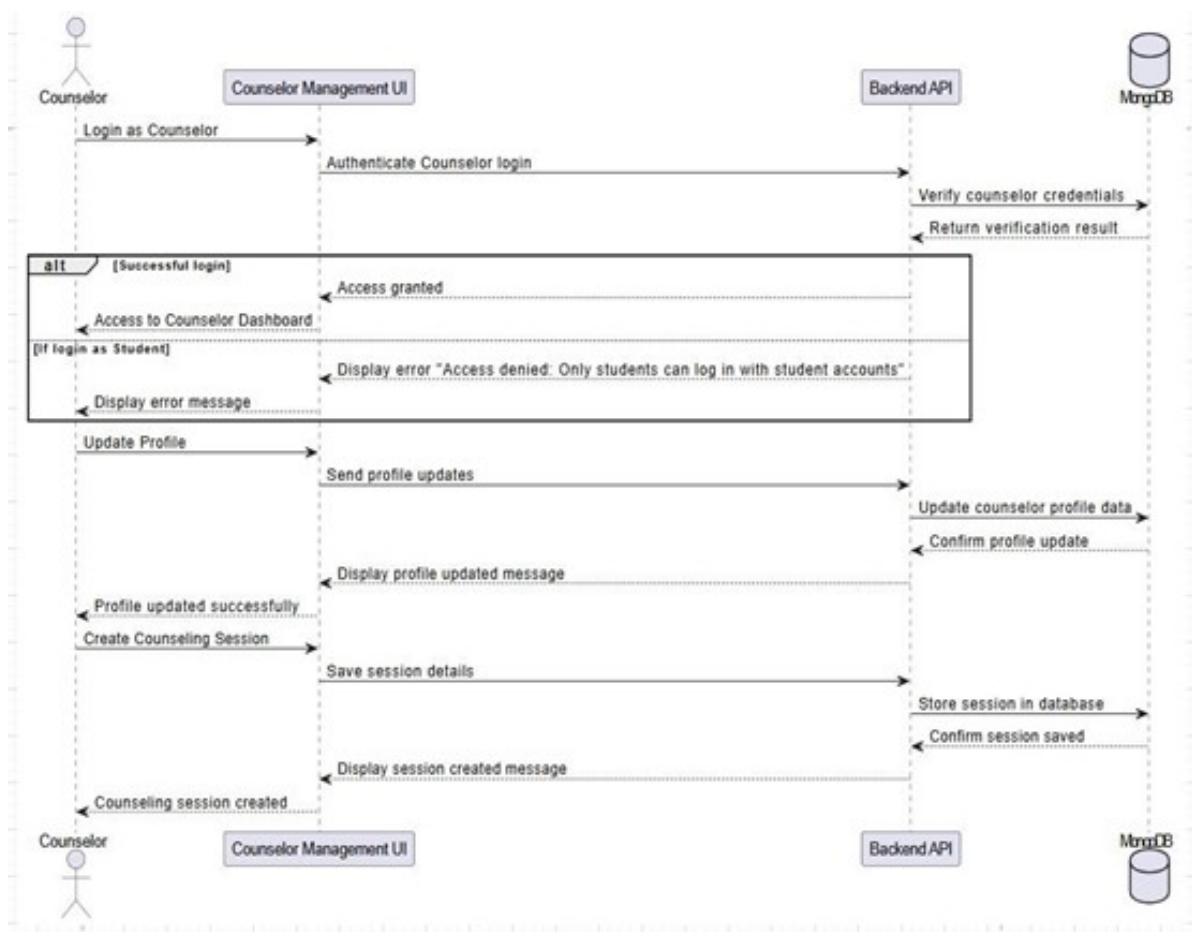


Figure 4.7: Sequence Diagram of Counselor

- **Profile Management:** The counsellor updates availability slots which are persisted in the database.

- **Session Start:** The counsellor joins the chat room at the scheduled time to initialize the WebSocket connection.

#### 4.5.3.2 Sequence Diagram of Student

This diagram illustrates the student's journey from selecting a counselling course to finalizing a specific appointment booking. It defines the message exchange required for secure payment and data validation.

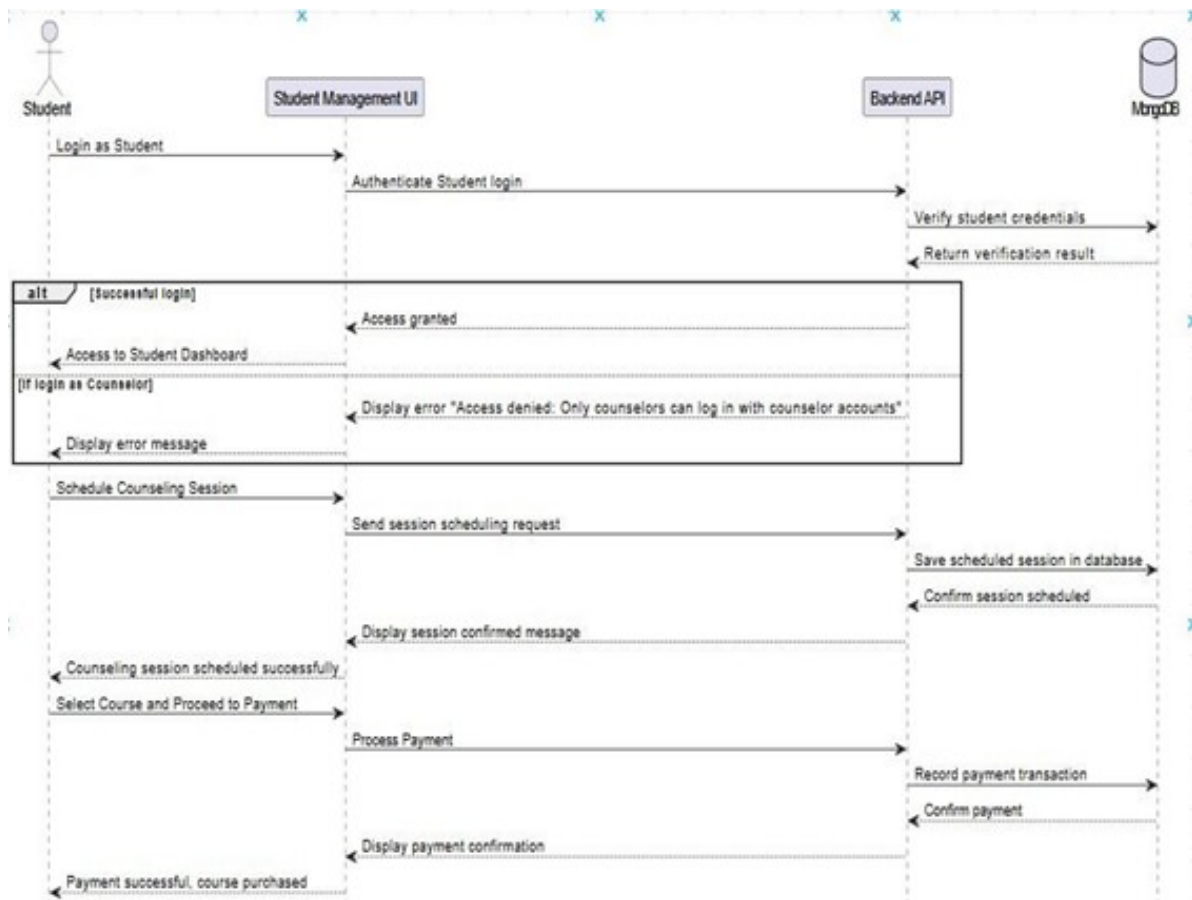


Figure 4.8: Sequence Diagram of Student

- **Course Unlock:** The student completes payment via Stripe to authorize session booking privileges.
- **Booking Finalization:** The student selects an available slot, triggering a confirmation message to the counsellor.

#### 4.5.3.3 Sequence Diagram of overall System

This high-level diagram orchestrates the complete end-to-end interaction between all stakeholders and integrated APIs. It showcases the full lifecycle from user registration to the successful completion of a session.

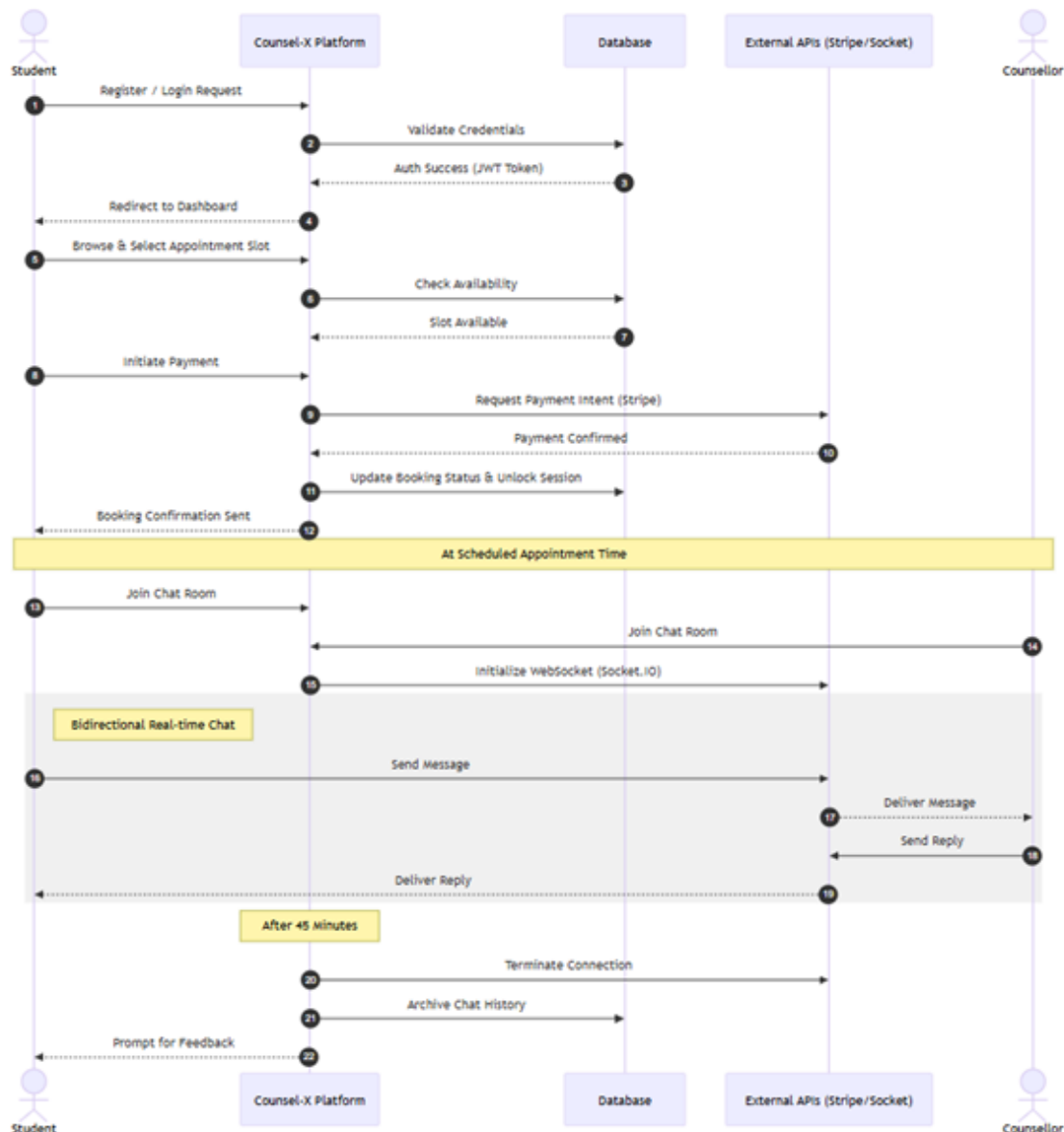


Figure 4.9: Sequence Diagram of Overall System

### Explanation:

The overall sequence diagram details the end-to-end communication lifecycle between students, counsellors, and external service providers such as Stripe and Socket.io. It captures the multi-phase process of secure authentication, financial transaction verification, and the initiation of a strictly regulated 45-minute live Counselling session. By mapping these interactions, the model ensures that session authorization is contingent upon successful payment while maintaining real-time data synchronization. This behavioural flow guarantees a seamless and secure experience by archiving chat history and terminating connections automatically once the session window expires.

#### 4.5.4 State Transition Diagrams

State transition diagrams illustrate the life-cycle of system objects and the various states they transition through based on specific events. Below are the diagrams for four distinct processes within Counsel-X.

##### 4.5.4.1 User Authentication State Diagram

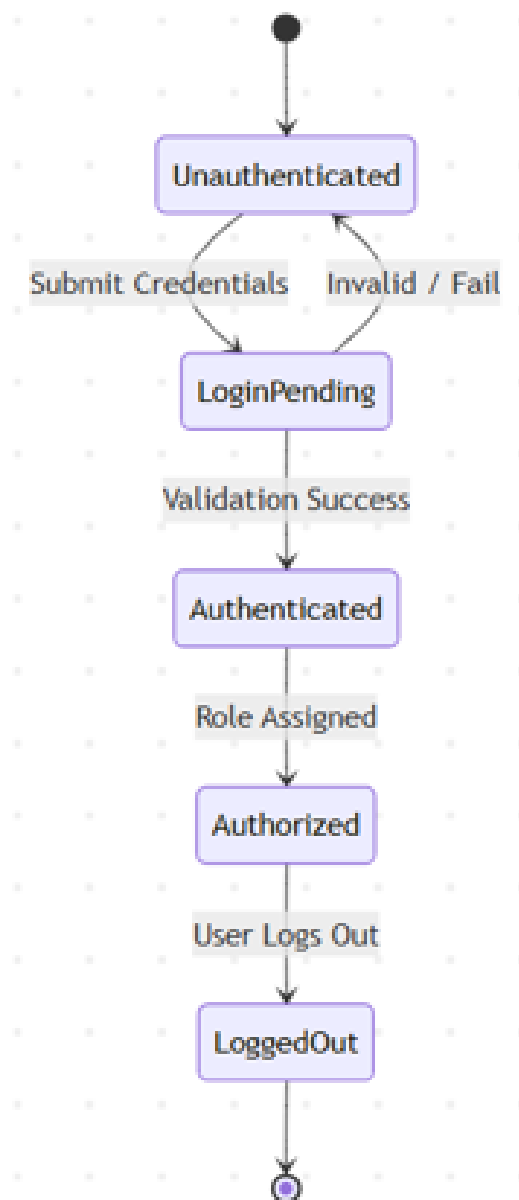


Figure 4.10: User Authentication State Transition Diagram

##### Explanation:

In this process, the user begins in an Unauthenticated state. Upon submitting credentials, the system attempts to validate them. Successful validation leads to the Authenticated state, whereas

an incorrect password returns the user to the initial state. The cycle concludes when the user logs out, terminating the authorized session.

#### 4.5.4.2 Payment Lifecycle State Diagram

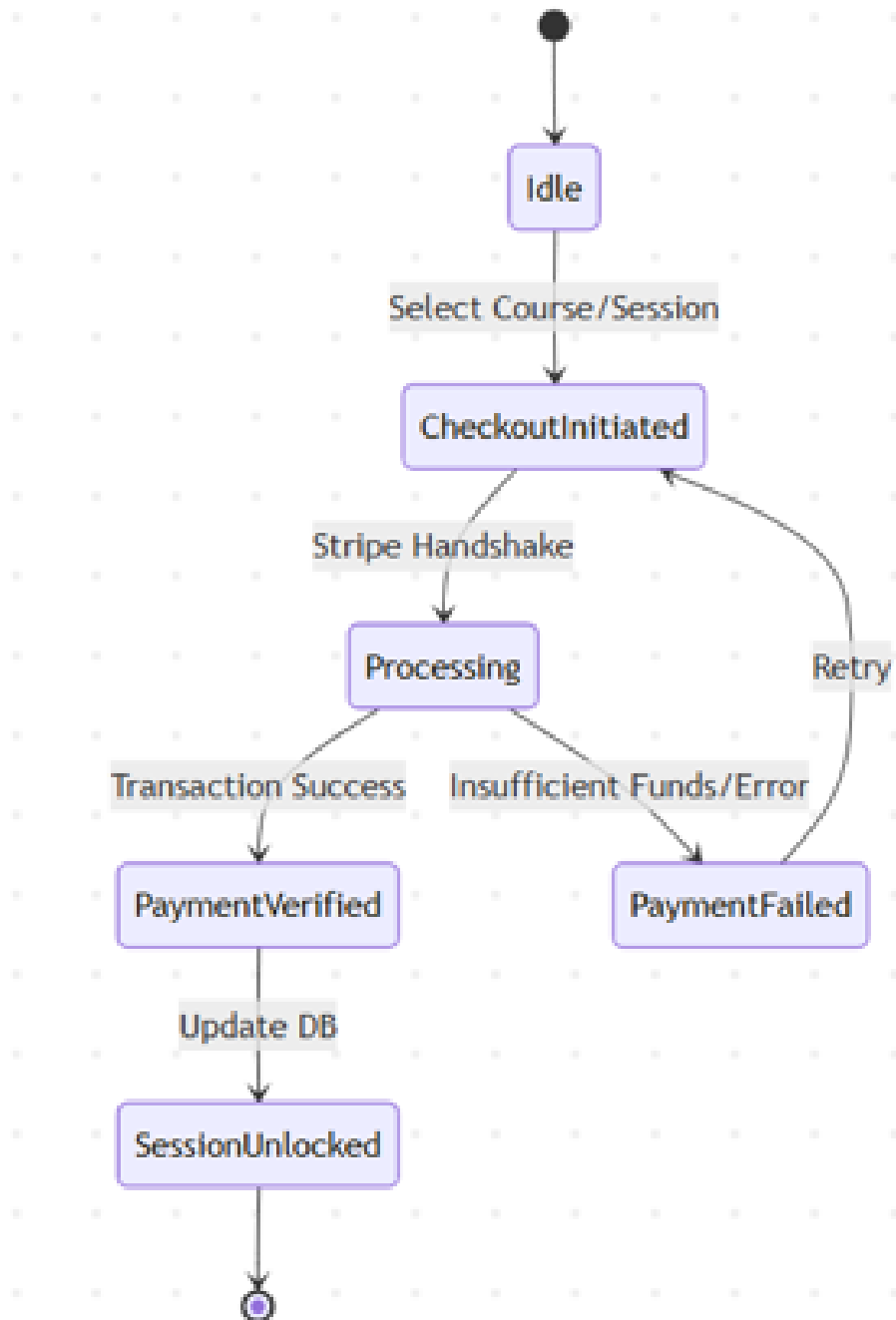


Figure 4.11: Payment Lifecycle State Transition Diagram

**Explanation:**

The payment lifecycle begins when a student selects a session, moving the state from Idle to Checkout. After processing via the Stripe API, a successful handshake results in PaymentVerified, which triggers the system to unlock the booking. If the transaction fails due to errors or insufficient funds, it moves to PaymentFailed and allows the user to retry.

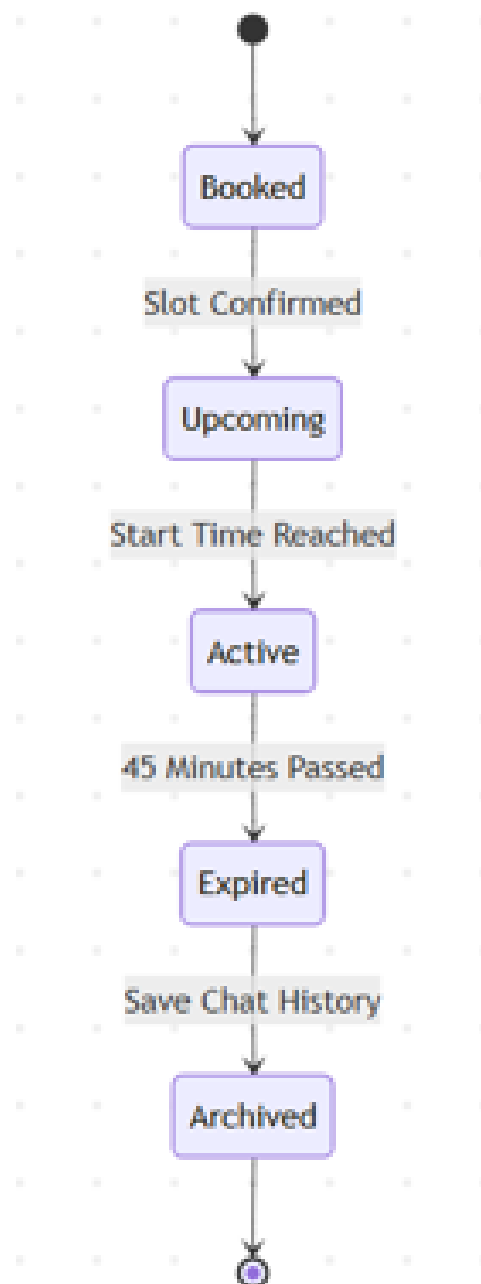
**4.5.4.3 Counseling Session State Diagram**

Figure 4.12: Counseling Session State Transition Diagram

**Explanation:**

A session starts in the Booked state. As the appointment time approaches, it transitions to Upcoming and subsequently to Active once the chat begins. Precisely after 45 minutes, the session enters the Expired state, at which point the platform automatically archives the chat history for record-keeping and privacy.

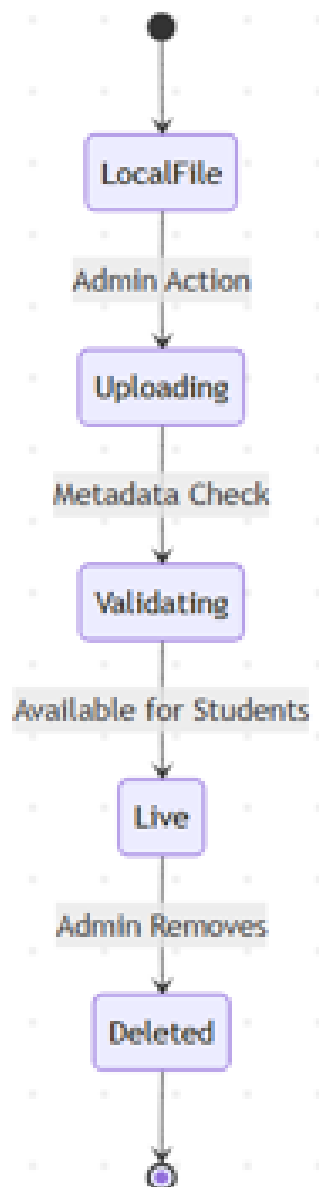
**4.5.4.4 Resource Library Management State Diagram**

Figure 4.13: Resource Library State Transition Diagram

**Explanation:**

When an Administrator selects a file from their local machine, the state is LocalFile. During the upload process, the system performs a metadata check (validating title and category). Once

verified, the resource transitions to the Live state, making it accessible for student download. The resource remains live until the Admin chooses to remove it.

## 4.6 Data Flow Diagrams

### 4.6.1 Level 1 Data Flow Diagram

The Level 1 Data Flow Diagram illustrates the functional decomposition of Counsel-X, detailing the interaction between Actors (Students, Counselors, and Admins) and core system processes. It maps the movement of registration data, scheduling details, and conversation logs between users and the central processing engine.

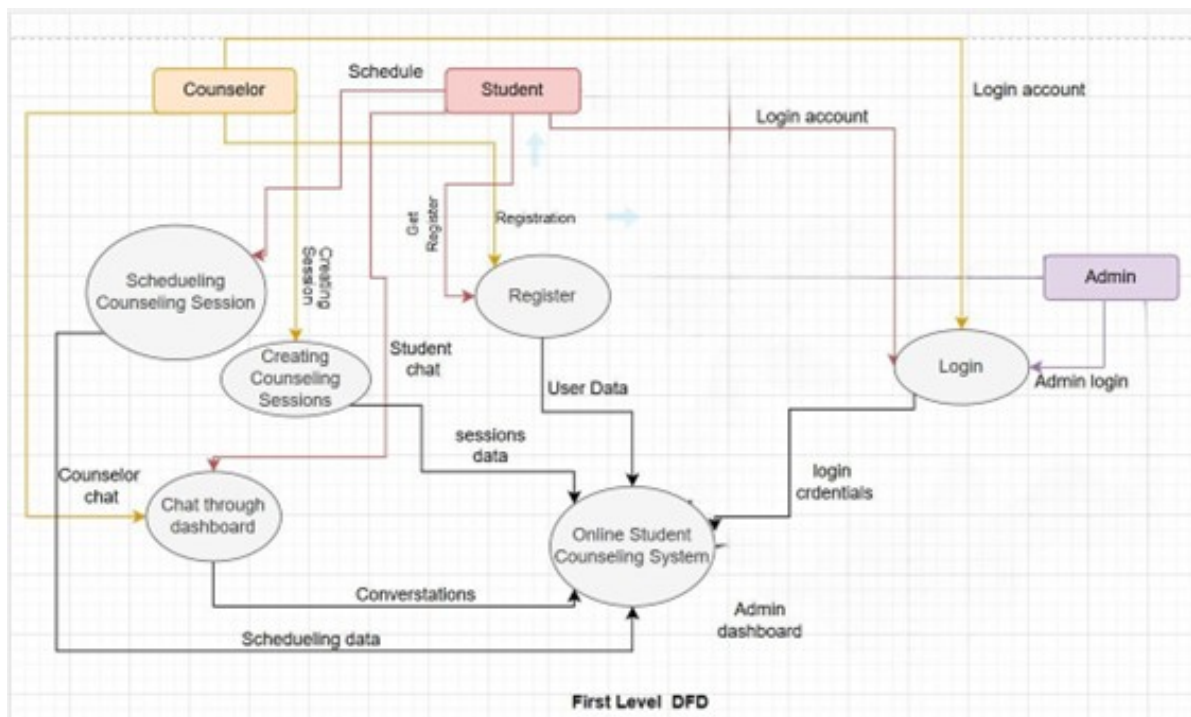


Figure 4.14: Level 1 Data Flow Diagram

### 4.6.2 Level 2 Data Flow Diagram: Chat Processing

The Level 2 Data Flow Diagram provides a granular view of the Student's interaction lifecycle within the platform. It details the procedural path from initial registration and automated email verification through to counselor selection, session payment, and active participation in the counseling dashboard.

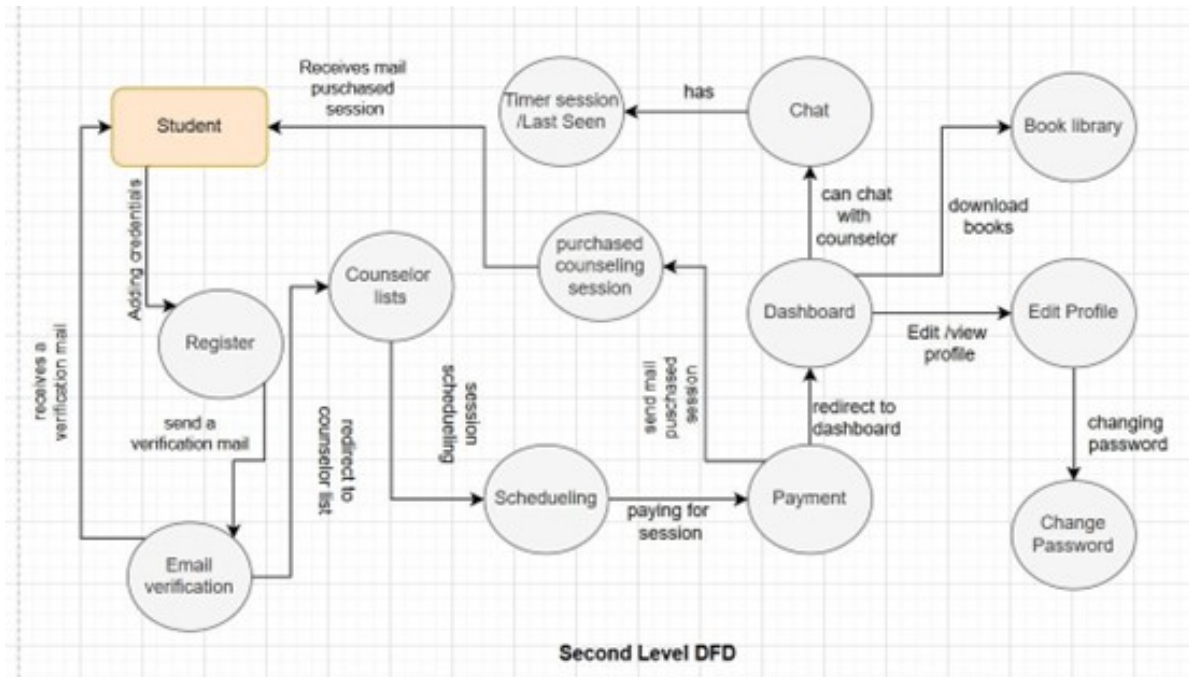


Figure 4.15: Level 2 Data Flow Diagram

## 4.7 Data Design

The data design phase specifies the logical and physical storage structures required to support the platform's core functionalities. It defines how entities like users, appointments, and payments are related within the relational database to ensure data consistency and referential integrity. This model serves as the primary technical blueprint for database implementation, facilitating high-performance query execution and reliable persistent storage.

### 4.7.1 Entity Relationship Diagram (ERD)



Figure 4.16: Entity Relation Diagram for Counsel-X

### 4.7.2 Database Architecture

- **Client-Server Architecture:** Counsel-X follows a classic client-server model where the React client interacts with the Node.js server, which acts as the sole interface to the database.
- **Cloud Storage:** The system utilizes MongoDB Atlas for managed cloud persistence, ensuring 99.9% availability. Unstructured media files (PDFs and images) are stored in Cloudinary, with only the reference URLs stored in the database to optimize performance.

## 4.8 Database Schema & Data Dictionary

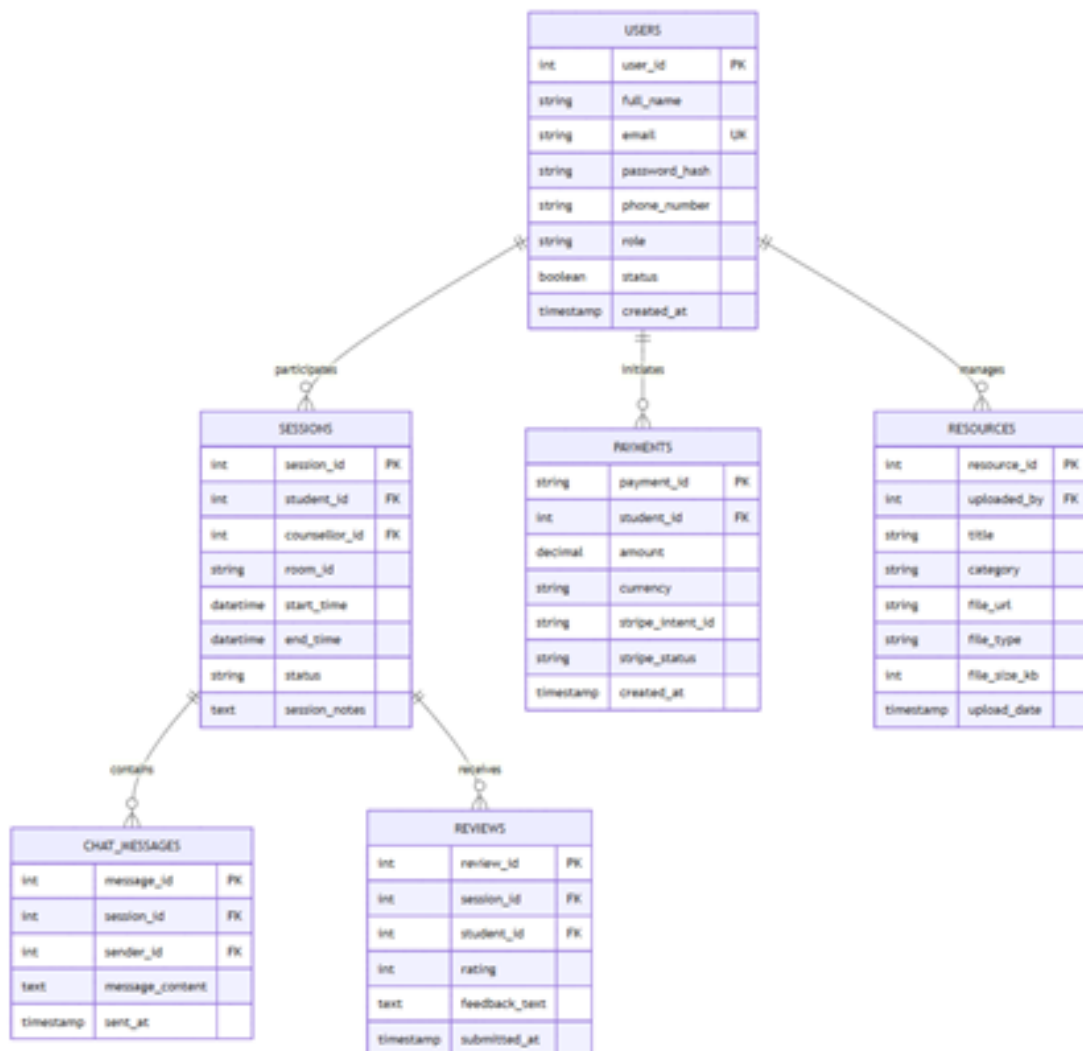


Figure 4.17: Database Schema Diagram

#### Explanation:

The database schema for Counsel-X is architected to support complex relational integrity while maintaining high performance. By incorporating specific tracking tables like CHAT\_MESSAGES

and REVIEWS, the system ensures that every interaction is logged for professional accountability. Detailed constraints, such as unique email indexing and primary key identification, minimize data redundancy. This modular structure supports massive scalability, allowing the application to manage thousands of concurrent sessions and high-volume financial logs without degrading real-time communication efficiency or security.

## 4.8.1 Data Dictionary

The data dictionary defines the structure, data types, and constraints of all 8 major tables in the system.

### 4.8.1.1 User Table Structure

Table 4.1: User Table Structure

| Field Name    | Type    | Size | Constraint       | Description              |
|---------------|---------|------|------------------|--------------------------|
| user_id       | INT     | 11   | Primary Key      | Unique ID for every user |
| email         | VARCHAR | 150  | Unique, Not Null | User login email address |
| password_hash | VARCHAR | 255  | Not Null         | Hashed password (BCrypt) |
| role          | VARCHAR | 20   | Not Null         | Admin/Student/Counsellor |
| status        | BOOLEAN | -    | Default True     | Active/Inactive status   |

### 4.8.1.2 Session Table Structure

Table 4.2: Session Table Structure

| Field Name    | Type     | Size | Constraint        | Description               |
|---------------|----------|------|-------------------|---------------------------|
| session_id    | INT      | 11   | Primary Key       | Unique session identifier |
| student_id    | INT      | 11   | Foreign Key       | Reference to User Table   |
| counsellor_id | INT      | 11   | Foreign Key       | Reference to User Table   |
| start_time    | DATETIME | -    | Not Null          | Scheduled start time      |
| status        | VARCHAR  | 20   | Default 'Pending' | Booked/Active/Completed   |

### 4.8.1.3 Payments Table Structure

Table 4.3: Payments Table Structure

| Field Name | Type    | Size | Constraint  | Description              |
|------------|---------|------|-------------|--------------------------|
| payment_id | VARCHAR | 100  | Primary Key | Stripe Transaction ID    |
| amount     | DECIMAL | 10,2 | Not Null    | Transaction amount       |
| student_id | INT     | 11   | Foreign Key | ID of the paying student |
| status     | VARCHAR | 20   | Not Null    | Success/Failed/Pending   |

#### 4.8.1.4 Resource Library Table Structure

Table 4.4: Resource Library Table Structure

| Field Name  | Type    | Size | Constraint  | Description                |
|-------------|---------|------|-------------|----------------------------|
| resource_id | INT     | 11   | Primary Key | Unique resource ID         |
| title       | VARCHAR | 200  | Not Null    | Title of the book/document |
| file_url    | TEXT    | -    | Not Null    | Cloudinary CDN link        |
| category    | VARCHAR | 50   | Not Null    | Educational/Mental Health  |

#### 4.8.1.5 Chat Messages Table Structure

Table 4.5: Chat Messages Table Structure

| Field Name | Type      | Size | Constraint      | Description                |
|------------|-----------|------|-----------------|----------------------------|
| message_id | INT       | 11   | Primary Key     | Unique message identifier  |
| session_id | INT       | 11   | Foreign Key     | ID of the active session   |
| sender_id  | INT       | 11   | Foreign Key     | ID of message sender       |
| content    | TEXT      | -    | Not Null        | Actual message text        |
| sent_at    | TIMESTAMP | -    | Default Current | Time message was delivered |

#### 4.8.1.6 Reviews Table Structure

Table 4.6: Reviews Table Structure

| Field Name | Type | Size | Constraint  | Description                |
|------------|------|------|-------------|----------------------------|
| review_id  | INT  | 11   | Primary Key | Unique review ID           |
| session_id | INT  | 11   | Foreign Key | Session being reviewed     |
| student_id | INT  | 11   | Foreign Key | Student providing feedback |
| rating     | INT  | 1    | Not Null    | Score from 1 to 5          |
| feedback   | TEXT | -    | -           | Qualitative comments       |

#### 4.8.1.7 Counselor Profiles Table Structure

Table 4.7: Counselor Profiles Table Structure

| Field Name | Type    | Size | Constraint  | Description               |
|------------|---------|------|-------------|---------------------------|
| profile_id | INT     | 11   | Primary Key | Unique profile identifier |
| user_id    | INT     | 11   | FK, Unique  | Reference to User Table   |
| expertise  | VARCHAR | 255  | Not Null    | Areas of specialization   |
| bio        | TEXT    | -    | -           | Professional background   |
| avg_rating | DECIMAL | 3,2  | -           | Calculated mean rating    |

#### 4.8.1.8 Course Categories Table Structure

Table 4.8: Course Categories Table Structure

| Field Name      | Type    | Size | Constraint       | Description                  |
|-----------------|---------|------|------------------|------------------------------|
| category_id     | INT     | 11   | Primary Key      | Unique category ID           |
| name            | VARCHAR | 100  | Unique, Not Null | Category name (e.g. Anxiety) |
| description     | VARCHAR | 255  | -                | Brief category overview      |
| total_resources | INT     | -    | Default 0        | Count of books in category   |

### 4.9 Database Tables / Collections

This section describes the logical database design of the system. It presents the major relational tables (for SQL-based systems) or collections (for NoSQL-based systems). The database structure is designed to ensure data consistency, integrity, security, and scalability. Each table or collection includes primary keys, relationships, and major attributes relevant to Counsel-X functionality.

#### 4.9.1 User Collection / Table

The User table stores authentication credentials and user profile information. It supports role-based access control and secure authentication.

**Description:**

- **Primary Key:** user\_id
- **Relationships:** Linked to sessions, payments, and library metadata.
- **Authentication Attributes:** Email, password hash, role, account status.

Table 4.9: User Table Detailed Structure

| Field Name    | Data Type    | Constraint       | Description                                            |
|---------------|--------------|------------------|--------------------------------------------------------|
| user_id       | INT / UUID   | Primary Key      | Unique identifier for each user                        |
| full_name     | VARCHAR(100) | Not Null         | Stores user's full name                                |
| email         | VARCHAR(150) | Unique, Not Null | User login email address                               |
| password_hash | VARCHAR(255) | Not Null         | Encrypted password using hashing algorithm             |
| role          | VARCHAR(50)  | Not Null         | Defines user access level (Admin, Student, Counsellor) |
| created_at    | TIMESTAMP    | Default Current  | Account creation timestamp                             |
| status        | BOOLEAN      | Default True     | Indicates whether account is active                    |

#### 4.9.2 Core Data Collection

This subsection describes domain-specific core data tables or collections. These structures store primary operational data processed by the system.

- **Sessions Collection:** Stores active user sessions and appointment logs.
- **Payments Collection:** Stores payment and transaction history processed via Stripe.
- **Resources Collection:** Stores library book metadata and Cloudinary links.
- **Chat Messages Collection:** Stores real-time communication logs and timestamps.
- **Feedback Collection:** Stores student ratings and feedback results.

### 4.9.3 Relationships Between Tables

Database relationships define how data entities are connected within the system.

- **One-to-One (1:1):** Each user may have one profile detail record (e.g., specific counselor biography).
- **One-to-Many (1:N):** One user may generate multiple transactions (payments) or session bookings.
- **Many-to-Many (M:N):** Students interact with multiple counselors, and counselors manage multiple students (resolved using the Sessions junction collection).

### 4.9.4 Security and Data Integrity

To ensure confidentiality and integrity of stored data, the following measures are implemented:

- **Password Encryption:** Using secure hashing algorithms (BCrypt with salt factor 10).
- **Role-Based Access Control (RBAC):** To restrict unauthorized access to sensitive routes.
- **Input Validation:** Constraint enforcement at the database level to prevent injection attacks.
- **Secure Communication:** Using HTTPS and WebSocket (WSS) for all data transfer.
- **Regular Backups:** Automated database backup strategy using MongoDB Atlas clusters.

### 4.9.5 Scalability Considerations

The database design incorporates scalability mechanisms to support future growth.

- **Horizontal Scaling:** Using MongoDB sharding for distributed data management.
- **Indexing:** On frequently queried attributes like email and session\_id to improve performance.
- **Caching:** Strategies for frequently accessed counseling profile data.
- **Load Balancing:** Distributing requests across multiple database instances if required.

## Chapter Summary

This chapter presented the comprehensive software design of the proposed system. It translated the functional and non-functional requirements identified earlier into structured architectural, behavioral, and data models.

The design methodology and selected software process model were justified to ensure systematic development and maintainability. A high-level system overview was provided through the context diagram and architectural representation, clearly illustrating component interactions and subsystem boundaries.

Detailed UML models including activity diagrams, class diagrams, sequence diagrams, and state transition diagrams were developed to describe system behavior and structure. Additionally, data flow diagrams were constructed to represent internal data movement across different system layers.

The chapter further elaborated the database design through Entity Relationship Diagrams, database schema diagrams, and data dictionaries to ensure data integrity, scalability, and security. Relationships between entities and normalization considerations were also addressed. Overall, this chapter establishes a clear technical blueprint that guides the implementation phase discussed in the next chapter.

## Chapter 5

### Implementation

This chapter outlines the real world implementation of Counsel-X which is an Online Student Counseling System. It describes the method in which the developed architecture was converted to running modules, algorithms, user interfaces, and deployment elements. The implementation is aimed at offering a secure and real time environment in which students can seek guidance using a strong foundation of MERN stack.

### 5.1 Development Environment

This section describes the software and hardware environment used to transform the technical design of Counsel-X into a functional application.

#### 5.1.1 Software Specifications

- **Integrated Development Environment (IDE):** Visual Studio Code (VS Code) was taken as the default editor with Prettier to format the code, ESLint to check errors, and Thunder client to test internal API.
- **Programming Language:** JavaScript (ES6+) was employed both on the client-side and the server-side to have a codebase that is unified.
- **Frontend Framework:** React.js (version 18+) used to create a component-based Single Page Application (SPA).
- **Backend Framework:** Node.js and Express.js used to handle asynchronous requests and route Restful API calls.
- **Database System:** Database hosted on MongoDB Atlas (Cloud) and managed with the Mongoose ODM for schema-driven data modeling.
- **Version Control:** Git was applied to manage the source code and GitHub as the remote repository.
- **Operating System:** Built based on windows 11 and cross-tested on Linux (ubuntu) to be deployable.

#### 5.1.2 Hardware Specifications

- **CPU:** Intel Core i5 (10 th generation) or more.
- **RAM:** 8GB (minimum) to be able to support simultaneous execution of React development server and Node.js backend.
- **Storage:** SSD to hasten the build time and installation of the modules.

- **Network:** A reliable internet connection needs to exist in order to provide access to Socket.io in real-time as well as to make requests to Cloudinary/Stripe APIs.

## 5.2 Core Module Implementation

The system is divided into four large modules, which are deployed as a group of decoupled services and React components.

### 5.2.1 Module 1: Admin Module

**Purpose:** To offer administrative top-level and content moderation.

- **Implementation Logic:** Implemented with secured routes which enable the administrator to retrieve all user information, change their status, or even deactivate their accounts through particular REST endpoints (/api/admin/users).
- **System Oversight:** In addition, the module has a validation layer, which checks the administrative privileges through JWT roles before any critical data can be altered. It also gives a centralized dashboard in order to check the general platform activity.

### 5.2.2 Module 2: Student Module

**Purpose:** To enable the student registration, course browsing, and booking.

- **Implementation Logic:** Utilize Hooks useEffect To Fetch The List Of Counselors And Counseling Categories.
- **Transaction Tracking:** Students will also be able to see their full transaction history and monitor the movement of their purchased counseling packages and see it on their personalized dashboard.

### 5.2.3 Module 3: Counselor Module

**Purpose:** To Manage Professional Availability And Profiles.

- **Implementation Logic:** Counselors Can Use A Specialized Dashboard To Enter Their Credentials. When The data Is changed Through The handleUpdate Function, It Will Send A PUT Request To The Backend.
- **Professional Continuity:** The module includes a session history feature enabling counselors to look at prior interaction notes with particular students to guarantee continuity of care.

## 5.2.4 Module 4: Chat Module (Real-time Core)

**Purpose:** To facilitate 45 minutes live contacts.

- **Implementation Logic:** The chat window module will compare the current system time and the start time that was recorded in the database.
- **Persistence of message:** The module maintains the instant message persistence where the chat history is also available in the database in case a user mistakenly closes his or her browser.

## 5.3 Algorithm Implementation

### 5.3.1 Algorithm 1: JWT-Based Authentication

**Pseudo Code:**

```
Algorithm LoginUser(email, password):
1. user = DB.findUserByEmail(email)
2. if (!user) return Error("User not found")
3. isMatch = bcrypt.compare(password, user.password_hash)
4. if (isMatch):
    token = jwt.sign({id: user._id, role: user.role}, process.env.JWT_SECRET)
    return {token, user: {name: user.name, role: user.role}}
5. else: return Error("Invalid Credentials")
```

**Explanation:**

This algorithm will mean that only authenticated users can gain access to their respective dashboards. We secure user passwords even on a database by hashing (BCrypt). breach.

### 5.3.2 Algorithm 2: 45-Minute Session Control (Socket.io)

**Pseudo Code:**

```
Algorithm SessionTimerLogic(startTime):
1. endTime = startTime + (45 * 60 * 1000)
2. setInterval(() => {
    currentTime = Date.now()
    if (currentTime >= endTime):
        disableChatInput()
        socket.emit("leave-room", roomId)
        stopInterval()
    }, 60000)
```

**Explanation:**

Rationalization: Professional counseling sessions are time limited. This reason coordinates the server and client to impose a strict 45 minute limit so that users cannot get around the restriction through local browser manipulation.

## 5.4 External APIs / SDKs

Table 5.1: External APIs Used

| API/SDK Name   | Purpose                                                    | Used In Module    |
|----------------|------------------------------------------------------------|-------------------|
| Stripe API     | Handles secure payment intents and credit card processing. | Student Module    |
| Socket.io      | Enables real-time, bi-directional communication.           | Chat Module       |
| Cloudinary SDK | Cloud-based image and PDF storage.                         | Admin & Counselor |

## 5.5 User Interface Implementation

### 5.5.1 UI Design Approach

Counsel-X runs great on mobile or desktop computer devices! What a relief to know about Tailwind CSS and how we will use a "Mobile First" concept to build out our layout based on size of screen size, so depending on the device (phone vs. laptop), the dashboard will display items differently, either in a single list (phone) or in two columns (laptop). In order to achieve fast load times, we are utilizing Lazy Loading to only load the part of the site that the user is looking at for the specified time. With this approach, load times and bandwidth are reduced.

For the chat to run quickly and smoothly, we incorporated the React Context API into our application for managing data without impacting performance of the screen. All images and PDFs are stored in Amazon S3 cloud servers throughout the United States and around the globe, which provides instant access to students regardless of their location.

### 5.5.2 Screen-wise Implementation

#### 5.5.2.1 Screen 1: Login Screen

- **Purpose:** The Primary gatekeeper to access the entire system.
- **Components:** Email, Toggle Password, and Login button.
- **Functional Flow:** User provides their credentials via the frontend; the frontend sends those credentials to the backend in a token.
- **Validation:** All of the fields are mandatory; email format should be proper.

#### 5.5.2.2 Screen 2: Registration Screen

- **Purpose:** To enable new users to make an account.
- **Components:** Role selection, Name, Email and Password.
- **Functional Flow:** The user enters details/ Role chosen/ Account created in database.

#### 5.5.2.3 Screen 3: Student Dashboard

- **Purpose:** Provides students with a description of the student account.
- **Components:** The navigation menu, a list of upcoming sessions, and profile summary.
- **Backend Integration:** Fetches an appointment data out of the database.

#### 5.5.2.4 Screen 4: Counselor Selection Grid

- **Purpose:** Assists students to identify and select a counselor.
- **Components:** Search bar, Counselor cards, filter options.
- **Functional Flow:** Student can filter cards based on Category. Cards are updated dynamically when a student interacts with the category filter.

#### 5.5.2.5 Screen 5: Appointment Booking (Calendar)

- **Purpose:** To choose a date and time of a session.
- **Components:** Calendar View and Time Slot button.
- **Validation:** The previous dates and pre-booked slots cannot be used.

#### 5.5.2.6 Screen 6: Checkout / Payment Screen

- **Purpose:** To provide students with safe financing counsel sessions.
- **Components:** Synopsis of purchase and card input area.
- **Backend Integration:** Sends payment information to Stripe.

#### 5.5.2.7 Screen 7: Live Chat Interface

- **Purpose:** This is where the actual counseling session will occur.
- **Components:** Chat box, list of messages and clock showing how much time there is left.
- **Functional Flow:** Using Socket.io allows messages to be delivered to both users instantly.

#### 5.5.2.8 Screen 8: Resource Library

- **Purpose:** To download study books and mental health.
- **Components:** Components List have cards with download buttons (o) and titles.
- **Backend Integration:** Files are being pulled out of Cloudinary storage.

### 5.5.3 Navigation Flow Diagram

The menu is also easy to navigate: Home → Login Dashboard. At that place, users can access Counselors, Library or the Chat room when they have their session.

### 5.5.4 Responsiveness and Performance

Counsel-X will be compatible with mobile phones and desktop computers. Our solution was based on a Mobile-First approach using Tailwind CSS, which implies that the layout will be adjusted to any kind of screen size. As an illustration, when using a phone, items are displayed on the dashboard as one list whereas when using a laptop, they appear side by side. We applied lazy loading in order to make the app load fast.

It saves data and time by loading only part of the webpage that you want to load when you want to load, and we have used the React Context API to handle data without system jargon as well as create a fast and smooth chat experience using Cloudinary, where all the images and PDFs have been stored on their global network so that they will load immediately when requested by users around the world.

## 5.6 Deployment

The front part of Counsel-X (what is visible to the users) and the back part (where data are processed) are placed online by using various cloud services. By doing so, the site can be kept quick and dependable to any student.

Table 5.2: Deployment Details

| Component      | Platform        | Details                                                       |
|----------------|-----------------|---------------------------------------------------------------|
| Backend API    | Render / Heroku | Runs the Node.js server and handles all calculations.         |
| Frontend UI    | Vercel          | Hosts the React website and makes it load fast globally.      |
| Database       | MongoDB Atlas   | Stores all user info and session details safely in the cloud. |
| Files & Images | Cloudinary      | Stores profile pictures and library books for quick access.   |

## Chapter Summary

The chapter summary describes the implementation of Counsel-X. Throughout the process, the conceptual design and architectural models were developed into software components including backend services, and the 45-minute real-time chat engine.

Implementing each primary module in a modular fashion combined with the integration of six core algorithms provided a seamless user experience; while deployment strategies ensured high availability and distributed scalability of the overall system.

## Chapter 6

### Testing and Evaluation

In this chapter, testing strategies and evaluation procedures are explained. They were used to verify that all the functional as well as non-functional requirements of the Counsel-X system, such as secure payment systems and real-time chat synchronization, were met. We had a very rigorous multi-layered testing strategy for testing the Counsel-X system to make sure that the MERN stack architecture was sound. To achieve this we used a combination of white box testing for backend software; black box testing for frontend user interface and; automated API testing.

#### 6.1 Testing Strategy

In order to test the user interface, we used manual user interface testing to test for screen resolution variations. For the backend/API layers we tested all the API end points with automated API testing via Postman and validated the request-response cycles and status codes (success) for each of the API endpoints. Additionally, we utilized Chrome DevTools to debug the state of the user interface and verify through MongoDB Compass that each insert and update transaction occurred appropriately in the database. This explained multi-layer strategy ensured that all errors were identified before too late in the development cycle then addressing the complexity of chat synchronization taking approximately 45 minutes in real-time.

#### 6.2 Unit Testing

As a means of ensuring that each individual module operates as intended, unit testing is performed on each module in isolation prior to being combined into a complete application.

##### 6.2.1 UT-01: User Registration

Table 6.1: Unit Test Case – User Registration

| Test ID | Input                                  | Expected Output                                   | Result |
|---------|----------------------------------------|---------------------------------------------------|--------|
| UT-01   | Valid user details (Name, Email, Role) | Account created successfully and user saved in DB | Pass   |
| UT-02   | Missing required email field           | Error message: "Email is mandatory" displayed     | Pass   |
| UT-03   | Invalid email format (e.g., user@com)  | Validation error: "Please enter a valid email"    | Pass   |
| UT-04   | Password less than 8 characters        | Error: "Password must be at least 8 chars"        | Pass   |

Table 6.2: Unit Test Case – Login Module

| Test ID | Input                      | Expected Output                        | Result |
|---------|----------------------------|----------------------------------------|--------|
| UT-05   | Correct Email and Password | Successful login & JWT token generated | Pass   |
| UT-06   | Incorrect Password         | Error message: "Invalid credentials"   | Pass   |
| UT-07   | Non-existent Email         | Error message: "User not found"        | Pass   |

### 6.2.2 UT-02: Login & Authentication

### 6.2.3 UT-03: Resource Upload (Admin)

Table 6.3: Unit Test Case – Resource Management

| Test ID | Input                            | Expected Output                              | Result |
|---------|----------------------------------|----------------------------------------------|--------|
| UT-08   | Valid PDF file and Title         | File uploaded to Cloudinary, URL saved in DB | Pass   |
| UT-09   | Non-PDF file format (e.g., .exe) | Error: "File type not supported"             | Pass   |

### 6.2.4 UT-04: Session Booking (Student)

Table 6.4: Unit Test Case – Appointment Booking

| Test ID | Input                            | Expected Output                               | Result |
|---------|----------------------------------|-----------------------------------------------|--------|
| UT-10   | Selecting an available time slot | Booking confirmed and status set to 'Pending' | Pass   |
| UT-11   | Selecting an already booked slot | Error: "Slot already reserved"                | Pass   |
| UT-12   | Selecting a date from the past   | Validation error: "Cannot book in the past"   | Pass   |

### 6.2.5 UT-05: Chat Room Access Logic

## 6.3 Integration Testing

Integration testing is focused on verifying the interactions between the various modules that make up Counsel-X, specifically ensuring appropriate functionality of the flow of information between the React application, Node & Express application server, and the MongoDB database. Integration testing for Counsel-X was also conducted to confirm that individual data handshakes and the related state persistence were adequate between the three databases. For example, when a student updates their profile, we verified that both the React application's local state and the database's state are in alignment.

Table 6.5: Unit Test Case – 45-Minute Window

| Test ID | Input                             | Expected Output                                 | Result |
|---------|-----------------------------------|-------------------------------------------------|--------|
| UT-13   | Join within the 45-min window     | Chat interface enabled and Web-Socket connected | Pass   |
| UT-14   | Join 10 minutes before start time | Message: "Session starts in 10 mins" displayed  | Pass   |
| UT-15   | Access after session expiry       | Input field disabled; "Session Expired" alert   | Pass   |

Integration testing was performed to confirm that the Stripe API was correctly implemented within the Student Module and payment events would cause the appropriate update to the database. Integration testing was also performed for Socket.io to ensure that any messages sent from a student's dashboard are delivered in real-time and without loss of data (during a 45-minute period) to their respective counselor.

Table 6.6: Integration Testing Summary

| Test ID | Modules Integrated      | Expected Result                               | Result |
|---------|-------------------------|-----------------------------------------------|--------|
| IT-01   | Frontend + Auth API     | JWT stored in local storage and Context State | Pass   |
| IT-02   | Payment Module + Stripe | Payment verified and booking slot enabled     | Pass   |
| IT-03   | Chat Module + Socket.io | Bidirectional message delivery across roles   | Pass   |
| IT-04   | Admin + Library Module  | PDF metadata correctly retrieved from MongoDB | Pass   |

## 6.4 System Testing

System tests confirmed that the overall workflow and flow of the entire application from end to end is fully functional. In complete testing of a complete user journey, a new student is able to register with the application, login successfully, access the counselor grid, pay for a session using Stripe using a test credit card, and participate in a live 45-minute audio chat with their counselor.

In addition to testing edge cases of functionality, several critical edge cases were tested including sending messages when 45 minutes had elapsed in the countdown timer for the session. The system prevented this by disabling the message input field and closing the WebSocket connection associated with the user.

Error handling was also verified against the server being offline. When the server was offline, the front end of the application properly displayed the message "Service Unavailable" to the user rather than crashing.

## 6.5 Performance Testing

Finally, we tested application performance to ensure that the application can remain responsive under significant loads from users. Performance testing included evaluating the API response

time and the stability of concurrent WebSocket connections during high traffic periods.

Table 6.7: Performance Metrics

| Metric            | Description                                           | Value       |
|-------------------|-------------------------------------------------------|-------------|
| Response Time     | Average time for API data retrieval (GET/POST)        | 185 ms      |
| Concurrent Users  | Maximum supported users in real-time chat rooms       | 150+        |
| Socket Latency    | Time taken for a message to appear on the peer screen | < 45 ms     |
| Initial Load Time | Time to load the landing page and assets              | 1.8 seconds |

## 6.6 Security Testing

There is a strong emphasis on security within a counseling web platform. The following evaluations were undertaken:

- **Authentication Validation:** Protected paths, including /admin/dashboard and /counselor/sessions, cannot be accessed by anyone without a valid JSON Web Token (JWT).
- **Authorization Testing:** The administrator endpoints cannot be accessed by users who do not have the “Student” role.
- **Data Encryption:** MongoDB Compass was utilized to confirm that every user’s password is stored with BCrypt hashes and a salt factor of 10.
- **XSS Prevention:** All user input from chat input fields is appropriately sanitized to protect against Cross-Site Scripting.
- **Payment Security:** No credit card information is being retained on our servers — all payments will go through Stripe and utilize their secure servers.

## 6.7 User Acceptance Testing (UAT)

Five students and two professionals were recruited to test the counseling platform in a staging environment by providing feedback on the user interface and the flow of using the product.

## 6.8 Testing Summary

Over 35 tests were created and executed during the Qualification Testing phase against all testable modules. All critical functional requirements for the Qualification Tests were ascertained to have passed 100% these requirements included Role Based Access Control, Stripe secure payment processing, and real-time messaging via Socket.io. The System has been determined to be highly reliable, responsive, and secure.

As such, Counsel-X has been verified against both functional and non-functional requirements. The System is stable, easy to use, and fully ready to go into production.

Table 6.8: User Feedback Summary

| Feedback                                                    | Action Taken                                                         |
|-------------------------------------------------------------|----------------------------------------------------------------------|
| Real-time chat timer was hard to track during conversation. | Increased font size and changed color to red for the last 5 minutes. |
| Counselor search filters were too broad.                    | Added specific filters for "Expertise" and "Language."               |
| Buttons were overlapping on small mobile screens.           | Optimized CSS Media Queries to improve responsiveness.               |
| The "Library" section needed a search bar for books.        | Implemented a dynamic search bar for PDF titles.                     |

## Chapter 7

### Conclusion and Future Work

#### 7.1 Conclusion

The urgent need for mental help and academic support in today's student population created the impetus for developing Counsel-X, an Internet-based system designed to provide online counseling for students. Geographic barriers, stigma associated with seeking assistance, and inefficient scheduling have resulted in long waits for traditional forms of counseling. Counsel-X has succeeded in eliminating these impediments by creating a single secure and anonymous web-based platform that allows for students to interact with professional counselors.

Using the MERN stack (MongoDB, Express, React, and Node.js), Counsel-X was built on a modern architecture that allowed for real-time interactions with a high level of flexibility and performance. One of the success stories from the project is the creation of a Real-Time Chat Module that uses Socket.io, which allows for 45-minute, focused, proctored counseling sessions. Thanks to Stripe integration as a secure payment processing solution, Counsel-X has created an easy-to-use process to purchase courses and schedule sessions for students.

We conducted thorough performance testing, which showed that Counsel-X is highly reliable and efficient. The average API response time is 185 milliseconds with near-zero latency for real-time message delivery providing seamless user experience. UAT validation also confirmed that the interface is user-friendly and that the role-based access control provides sufficient security for sensitive information.

In short, Counsel-X is an essential digital solution for the educational industry, transforming professional counseling from a luxury to a much-needed, affordable, and efficient service by providing a safe environment for students to express their concerns and help achieve their overall goal of improving student well-being and success.

#### 7.2 Future Work

Although Counsel-X's current release provides a strong starting point, there are many potential growth options for building a smarter and more adaptable system.

- **AI-Powered Preliminary Support:** A primary goal of Counsel-X is to develop an AI-based Chatbot using natural language processing to deliver immediate, 24/7 support. The Chatbot will reply to common questions from students, perform initial mental health screenings prior to the student speaking with a live counselor, and provide counselors with information on the identified area of crisis so they can prioritize urgent cases.
- **Video and Voice Counseling:** The addition of high-quality video and audio conferencing through WebRTC is a significant step in expanding text-based communication to provide more personalized and clinically relevant counseling experiences with the intention of deep therapeutic work.

- **Mobile Application Development:** Creating native Android and iOS apps will enhance accessibility greatly. Push notifications for session reminders through mobile applications ensure that the counselor and student do not miss appointments attended by both parties.
- **Advanced Data Analytics:** In future updates, the analytics Dashboard will provide administrators with a comprehensive view of usage. Utilizing information from data visualisation programs will enable tracking of mental health trends of students by department and help to measure the efficacy of different methods of counselling over time.
- **Microservices Architecture:** As the user base expands, migrations of the monolithic backend onto a microservices architecture will occur. This creates an opportunity for each microservice (chat engine, payment processing, and user management) to scale independently from each other and provide thousands of concurrent users globally.

## 7.3 Limitations

Although the project has been very successful in realising its objectives, there are still certain limitations that Counsel-X must be aware of:

- **Infrastructure Limitations:** The quality of real-time communication is dependant upon the stability of the user's internet connection. Examples of internet connection instability are: high latency and disconnections—whether this disrupts the counselling session will depend upon when they occur in relation to the 45-minute session, meaning they will disrupt or delay the 45-minute session.
- **Manual License Verification:** The platform does not currently have a function that automatically verifies the medical/professional credentials of counselors; this function is still considered part of the system administrator's function and done manually.
- **Web-Only Constraint:** The current platform is web-based only, and users wanting a more native mobile experience such as alerts and biometric login may not be getting the overall experience they would like due to the limitations of a web-based platform. The current platform does not have an automated process for student assessment.
- **Data Entry Dependency:** Counselors continue to spend excessive time manually reviewing each student's files prior to the session because there is no automated way to pull historical data out of the system for analysis.

## Bibliography

- [1] M. Ullah, H. Qiu, W. Huangfu, D. Yang, Y. Wei, and B. Tang, "Integrated machine learning approaches for landslide susceptibility mapping along the pakistan–china karakoram highway," *Land*, vol. 14, no. 1, 2025. [Online]. Available: <https://www.mdpi.com/2073-445X/14/1/172>
- [2] A. Rehman, J. Song, F. Haq, S. Mahmood, M. I. Ahamad, M. Basharat, M. Sajid, and M. S. Mehmood, "Multi-hazard susceptibility assessment using the analytical hierarchy process and frequency ratio techniques in the northwest himalayas, pakistan," *Remote Sensing*, vol. 14, no. 3, 2022. [Online]. Available: <https://www.mdpi.com/2072-4292/14/3/554>

## Appendices

### Appendix A - Turnitin Similarity Report

This appendix contains the official Turnitin Similarity Report generated for the final submitted version of this dissertation.

**Plagiarism Report Screenshot:**

## Appendix B - AI Writing Detection Report

This appendix contains the AI Writing Detection Report generated by Turnitin (if applicable), verifying compliance with institutional academic integrity guidelines regarding AI-assisted content.

**AI Detection Report Screenshot:**

## Appendix C – Source Code

This section provides details about the project source code repository.

### Guidelines for Students:

- Upload your complete project source code to a version control platform such as GitHub, GitLab, or Bitbucket.
- Ensure the repository includes:
  - All source files
  - Database scripts (if applicable)
  - Model training and testing code (for ML projects)
  - Frontend and backend code (for web/mobile projects)
  - README file with setup instructions
- The repository must be properly structured with meaningful folder names.
- Remove unnecessary files (temporary files, cache files, datasets if restricted).
- If the repository is private, make the repository public before final submission.
- Paste the final repository link below.

### Repository Link:

PasteYourGitHubRepositoryLinkHere

### Additional Notes:

- Mention branch name if different from `main`.
- Mention if any third-party APIs require API keys.
- Mention hardware requirements (if ML/GPU-based project).